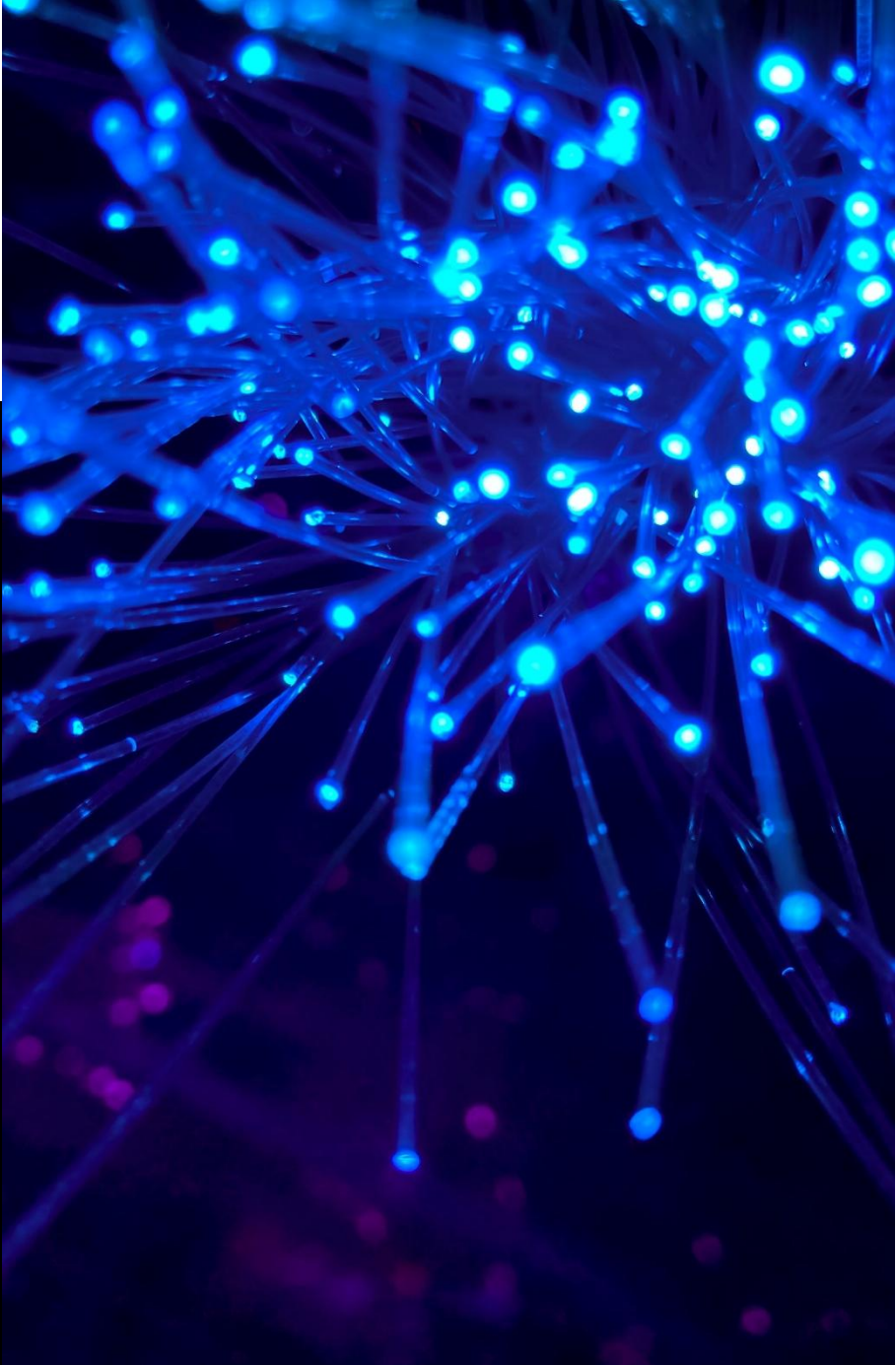




Gestión de Datos

Almacenamiento de datos I: SQL

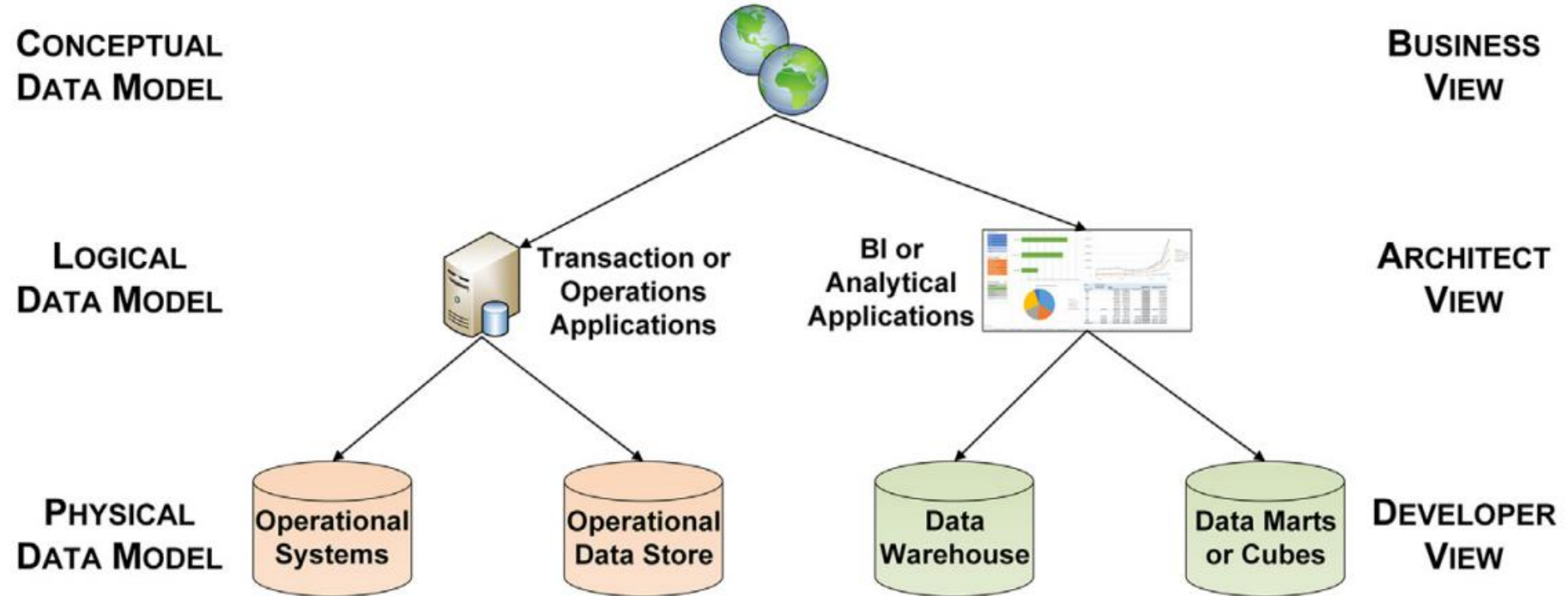


Fundamentos de Modelización de Datos

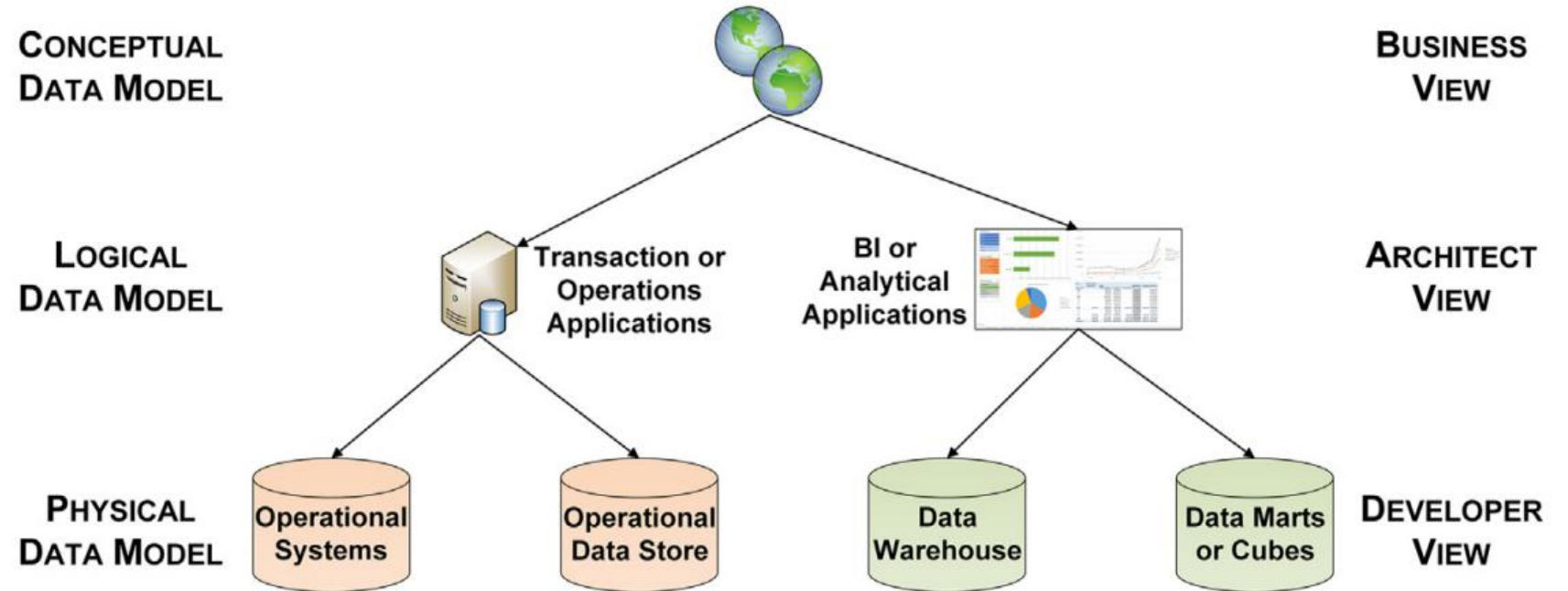
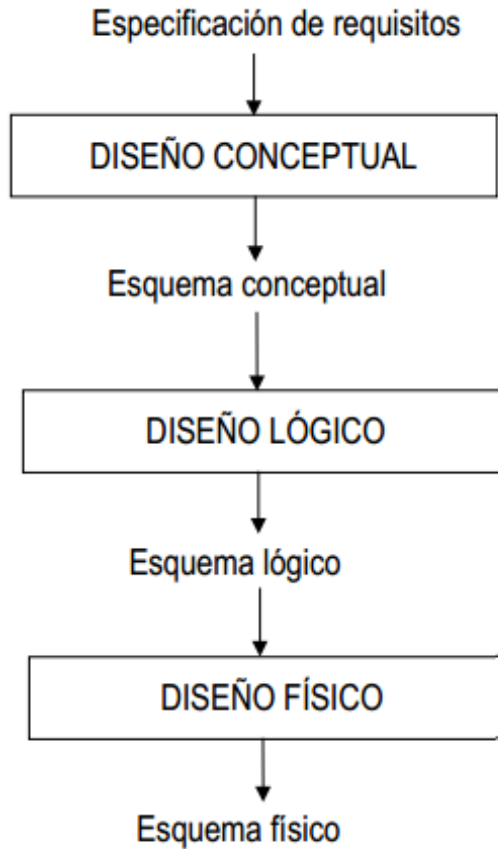
Es un enfoque estructurado para identificar y analizar los componentes de datos de las especificaciones del sistema de información.

Implica un estrecho trabajo entre los grupos de negocio y de IT, y el cumplimiento de un enfoque de mejores prácticas para garantizar que el modelo de datos sea el más apropiado para crear la aplicación.

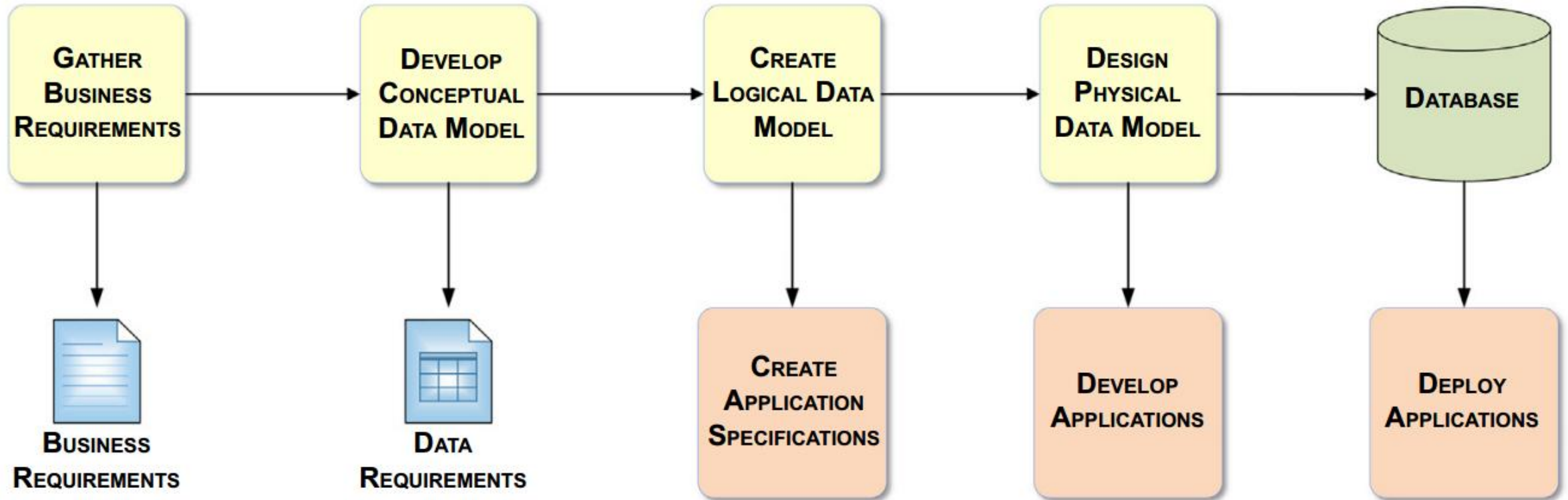
Modelado de datos

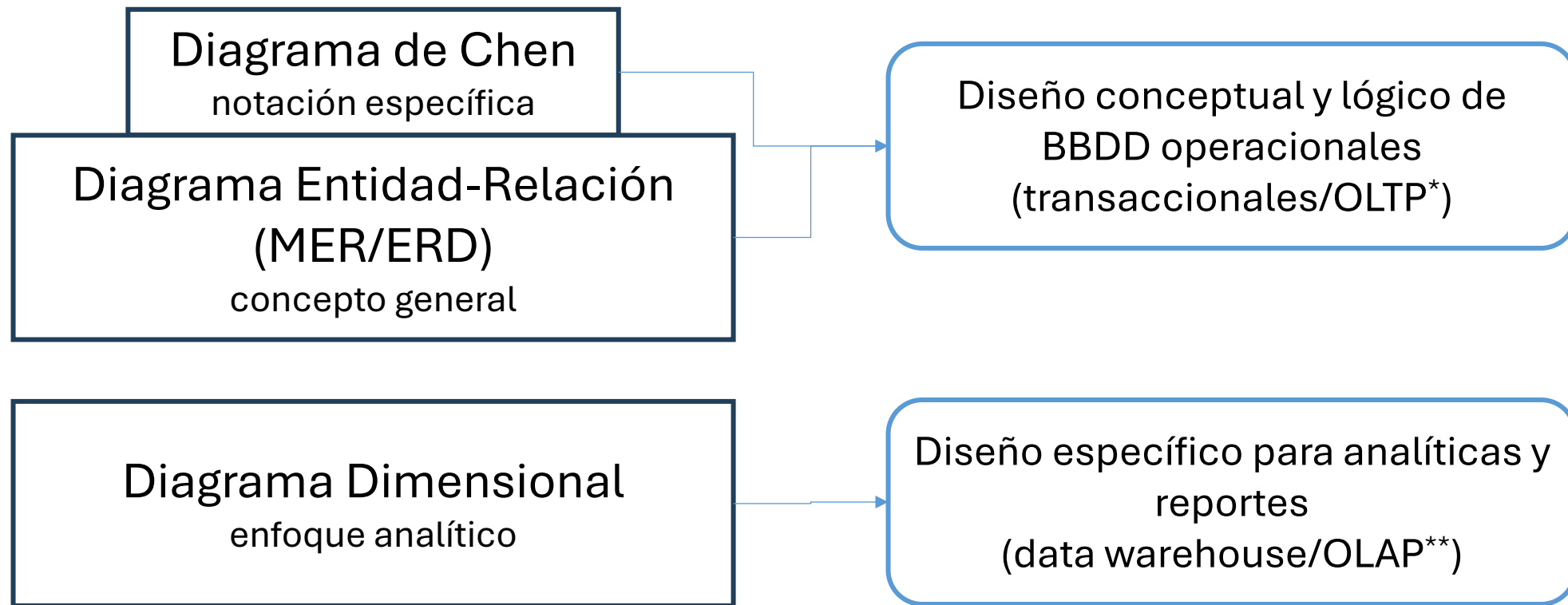


Modelado de datos



Modelado de datos





*OLTP (*On-Line Transactional Processing* o Procesamiento de Transacciones en Línea)

**OLAP (*On-Line Analytical Processing* o Procesamiento Analítico en Línea)

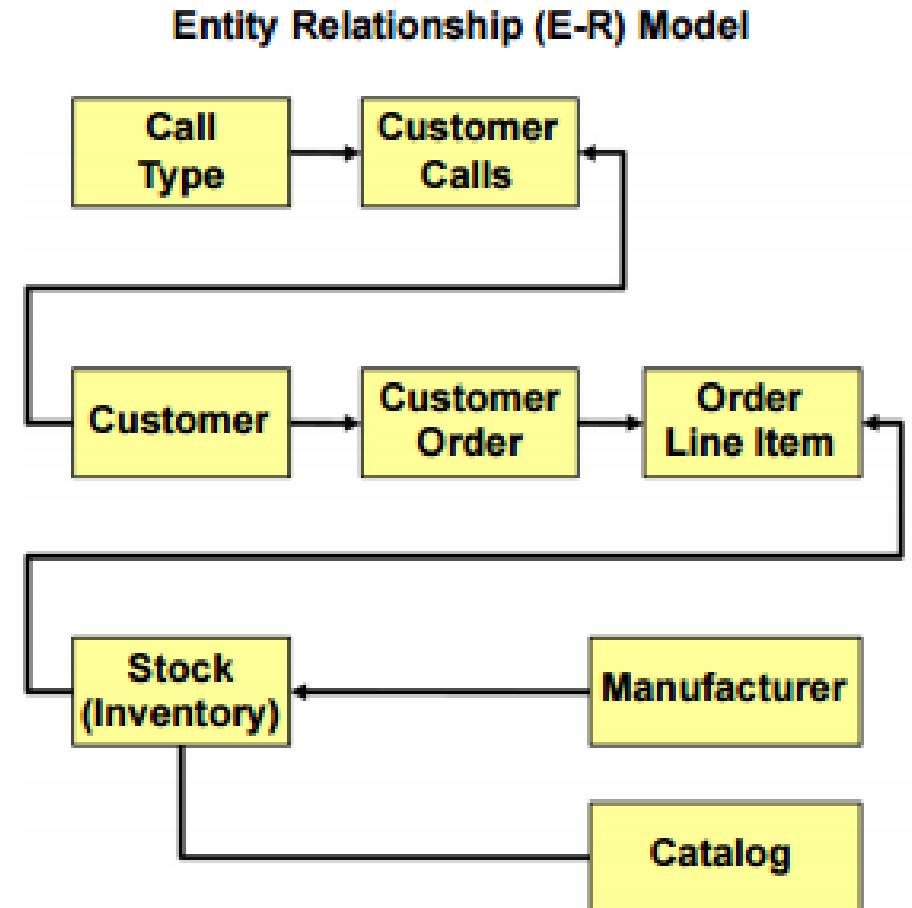
Modelado de datos

Característica	Diagrama de Chen	Diagrama E-R	Diagrama Dimensional
Propósito	Diseño conceptual	Diseño de BBDD operacionales	Diseño analítico / data warehouse
Enfoque	Académico, formal	Técnico, funcional	Negocio
Estructura	Entidad-Relación-Atributo	Tablas normalizadas	Hechos y dimensiones (estrella o copo de nieve)
Normalización	Alta	Alta	Baja
Uso	Modelo inicial	OLTP	BI / OLAP

Modelado de datos

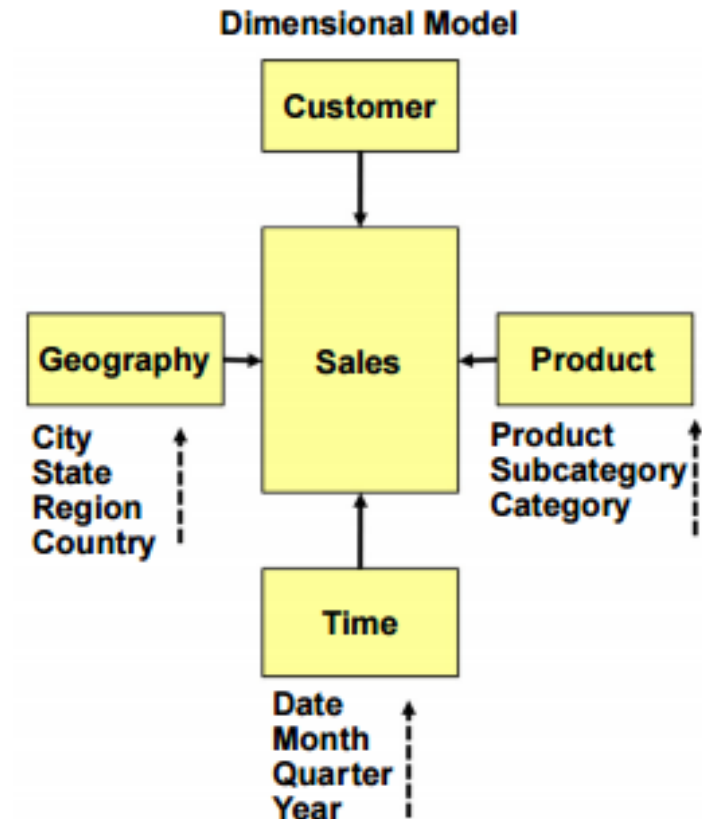
El **modelo entidad-relación (ER)** es una técnica de modelado lógico de datos que representa los objetos del negocio (entidades), sus características (atributos) y las conexiones entre ellos (relaciones).

- Estructura basada en entidades y relaciones: representa la información en términos de entidades (objetos del mundo real) y sus relaciones.
- Uso de claves primarias y foráneas: se utilizan identificadores únicos para garantizar la integridad referencial.
- Normalización: se busca minimizar la redundancia de datos aplicando.
- Uso en bases de datos transaccionales: es el modelo que se usa para almacenar datos en sistemas como ERP, CRM, sistemas de facturación, etc.
- Alta cantidad de tablas y relaciones complejas: puede ser difícil de consultar directamente para análisis de negocio.

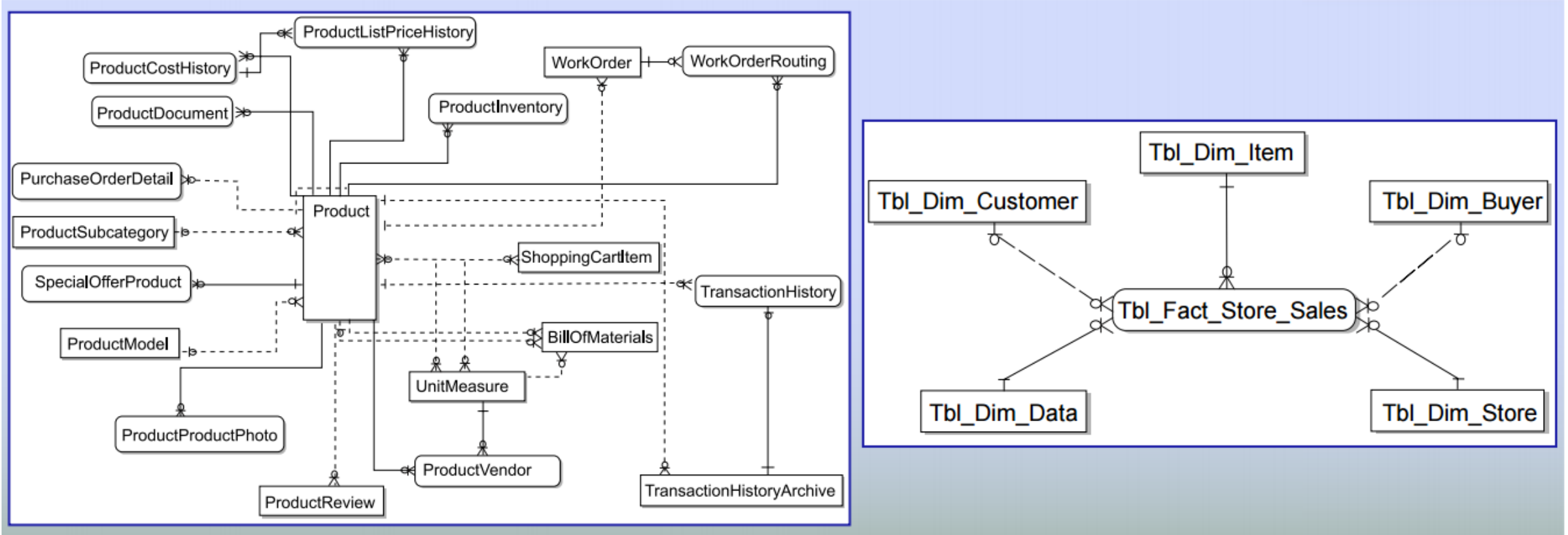


El **modelo dimensional** es una técnica de modelado diseñada específicamente para aplicaciones de inteligencia de negocio (BI) y almacenamiento de datos (Data Warehousing). Su objetivo es optimizar el análisis y la consulta de grandes volúmenes de datos.

- Estructura basada en hechos y dimensiones: se organiza en torno a tablas de hechos (datos cuantificables) y tablas de dimensiones (contexto de análisis).
- Optimización para consultas y reportes: facilita el análisis rápido y eficiente de datos empresariales.
- Desnormalización: se sacrifica la normalización para mejorar la velocidad de consulta, permitiendo redundancia controlada.
- Utilizado en sistemas de análisis (OLAP, *On-Line Analytical Processing*): en almacenes de datos y herramientas de BI para *reporting* y *dashboards*.
- Menos tablas y estructura más sencilla: permite consultas más intuitivas y rápidas para el usuario final.



Modelado de datos

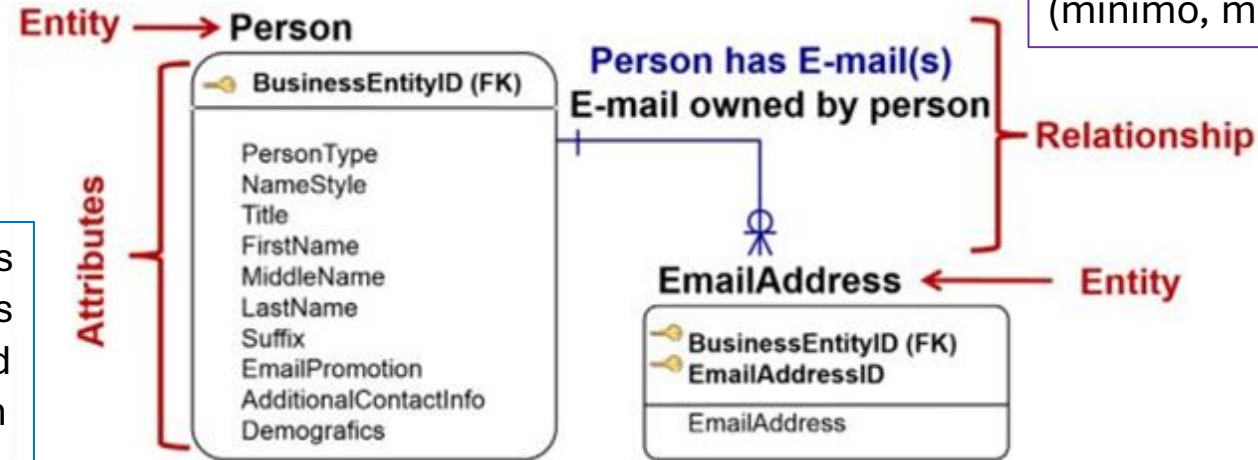


Modelo ER

Una **entidad** es un objeto o concepto del mundo real sobre el que se desea almacenar información en la base de datos. Puede representar personas, lugares, eventos o cosas que tienen relevancia en el sistema.

Una **relación** representa la asociación entre dos o más entidades en una base de datos. La cardinalidad define el número de ocurrencias que puede tener una entidad en relación con otra. Se expresa como (mínimo, máximo).

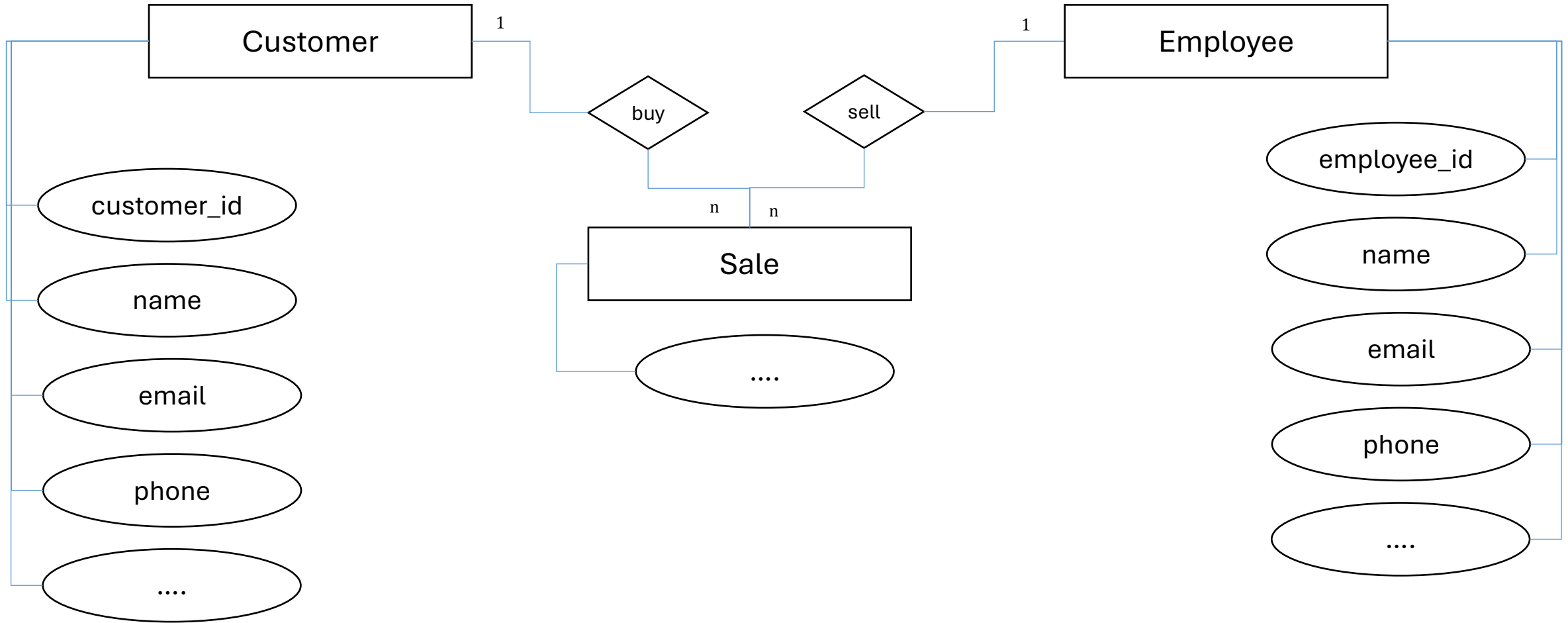
Los **atributos** representan las propiedades o características de una entidad. Cada entidad tiene atributos que describen su información.



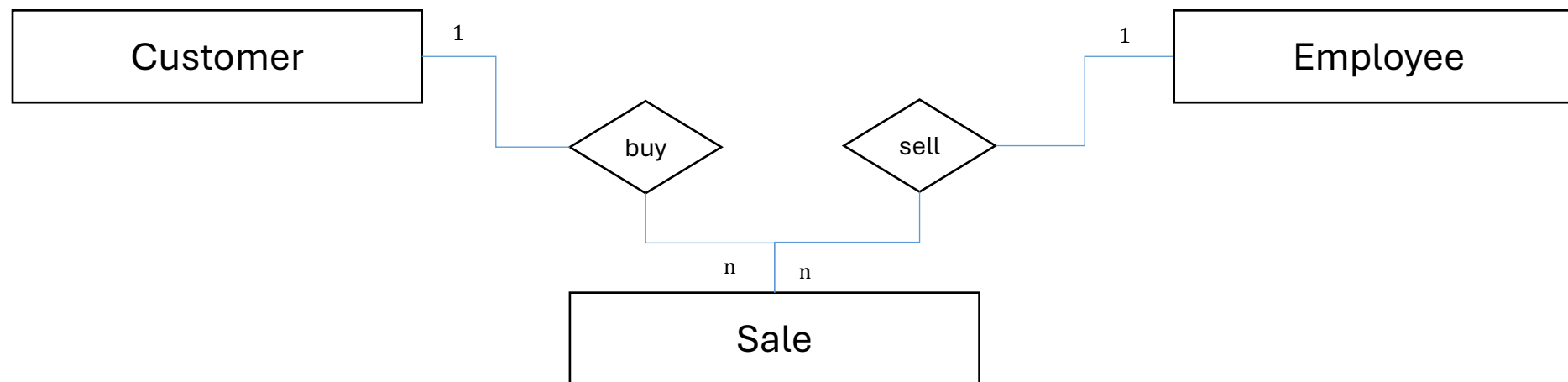
La **Clave Primaria (PK)** es un atributo (o conjunto de atributos) que identifica de manera única cada registro en una entidad. No puede tener valores nulos ni repetidos.

Una **Clave Foránea (FK)** es un atributo en una tabla que referencia a la clave primaria de otra tabla. Se usa para establecer relaciones entre entidades.

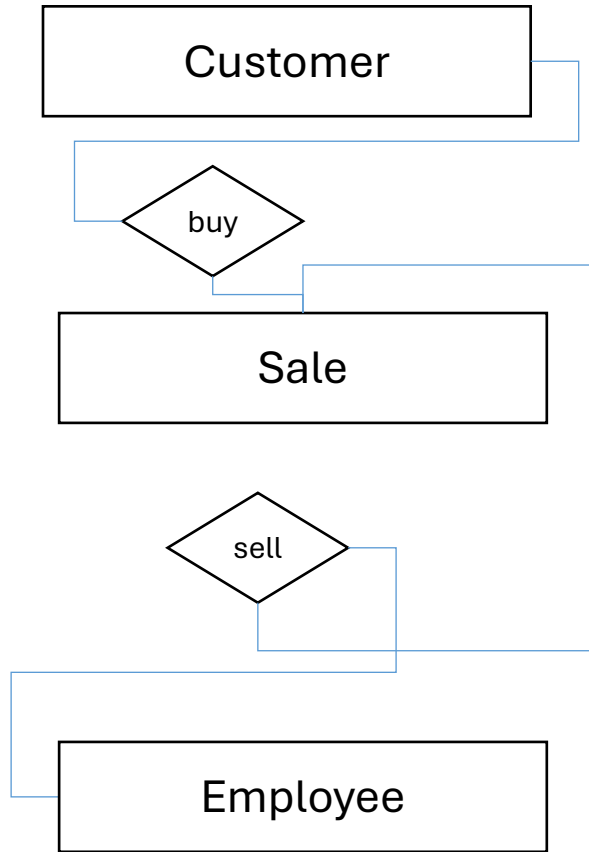
Modelo ER



Modelo ER



Modelo ER



employee	
employee_id	bigint
first_name	varchar(80)
last_name	varchar(120)
email	varchar(190)
phone	varchar(40)
role	varchar(20)
active	boolean
hired_at	date
created_at	timestamp
updated_at	timestamp

customer	
customer_id	bigint
name	varchar(150)
email	varchar(190)
phone	varchar(40)
billing_address	varchar(250)
shipping_address	varchar(250)
created_at	timestamp
updated_at	timestamp

sale	
sale_id	bigint
customer_id	bigint
employee_id	bigint
sale_date	datetime
status	varchar(20)
subtotal	decimal(14,2)
tax_total	decimal(14,2)
total_amount	decimal(14,2)
notes	varchar(255)

Modelo ER

- **Entidad fuerte (independiente):** No depende de otra entidad para existir. Ejemplo: "Cliente".
- **Entidad débil (dependiente):** No puede existir sin una entidad fuerte asociada. Ejemplo: "DetallePedido" depende de "Pedido".

Two Types Of Entities:

Independent

Dependent

SalesOrderHeader

🔑 SalesOrderID

SalesOrderDetail

🔑 SalesOrderID (FK)
🔑 SalesOrderDetailID



Two types of attributes:

Key

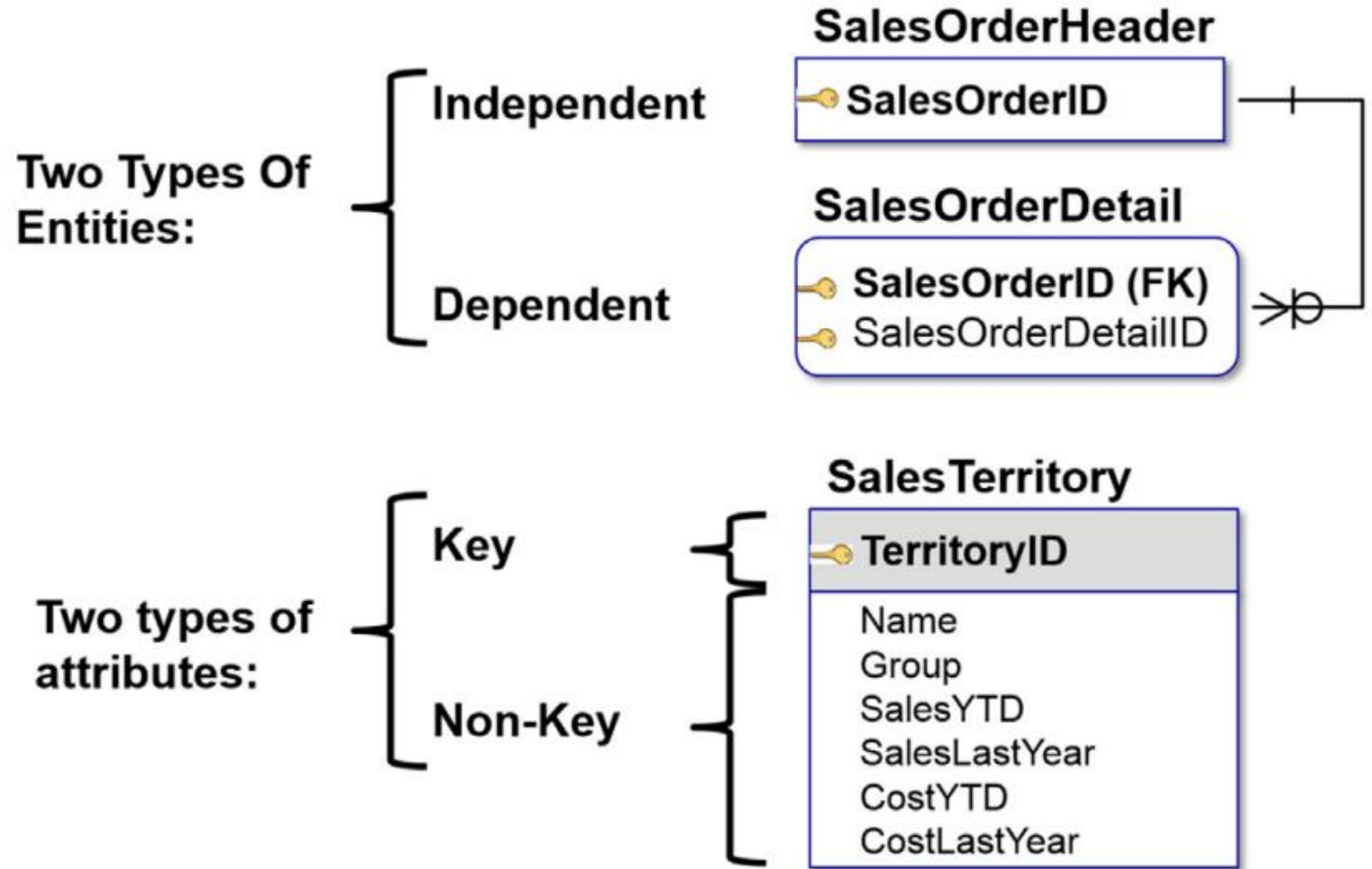
Non-Key

SalesTerritory

🔑 TerritoryID

Name
Group
SalesYTD
SalesLastYear
CostYTD
CostLastYear

- **Atributo Clave (Key Attribute):** Identifica de manera única a cada entidad.
- **Atributo Simples:** No pueden dividirse en subpartes (Ej: DNI).
- **Atributo Compuesto:** Se puede dividir en subpartes (Ej: Nombre completo → Nombre, Apellido).
- **Atributo Derivado:** Se obtiene de otros atributos (Ej: "Edad" derivado de "Fecha de Nacimiento").
- **Atributo Multivalor:** puede tener más de un valor por registro (Ej: teléfono, títulos universitarios, colores de un producto)



Cardinalidad - Notación de pata de gallo

Se utilizan tres símbolos para representar la cardinalidad:

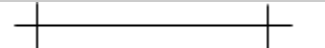
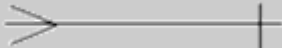
- El **anillo** representa el "cero"
- El **guion** representa "uno"
- La **pata de gallo** representa "muchos" o "infinito"



Estos símbolos se utilizan en pares para representar los cuatro tipos de cardinalidad que una entidad puede tener en una relación. El componente interno de la notación representa el mínimo y el externo, el máximo.

- anillo y guion → mínimo cero, máximo uno (opcional)
- guion y guion → mínimo uno, máximo uno (obligatorio)
- anillo y pata de gallo → mínimo cero, máximo muchos (opcional)
- guion y pata de gallo → mínimo uno, máximo muchos (obligatorio)

Modelo ER

Tipo de Cardinalidad	Descripción	Ejemplo	Dibujo
Uno a Uno (1:1)	Cada entidad en un conjunto está relacionada con solo una entidad del otro conjunto.	Un pasaporte solo pertenece a una persona.	
Uno a Muchos (1:M)	Una entidad en un conjunto está relacionada con muchas entidades del otro conjunto.	Un cliente puede hacer muchos pedidos, pero un pedido solo pertenece a un cliente.	
Muchos a Uno (M:1)	Muchas entidades en un conjunto están relacionadas con una sola entidad en el otro conjunto.	Muchos empleados trabajan en un departamento, pero cada empleado pertenece a un solo departamento.	
Muchos a Muchos (M:M)	Muchas entidades en un conjunto pueden estar relacionadas con muchas entidades en el otro conjunto.	Un estudiante puede inscribirse en varios cursos, y un curso puede tener muchos estudiantes.	

Relación identificativa o identificadora (línea continua): la entidad hija no puede existir sin la entidad padre y **depende de ella para ser identificada** unívocamente.

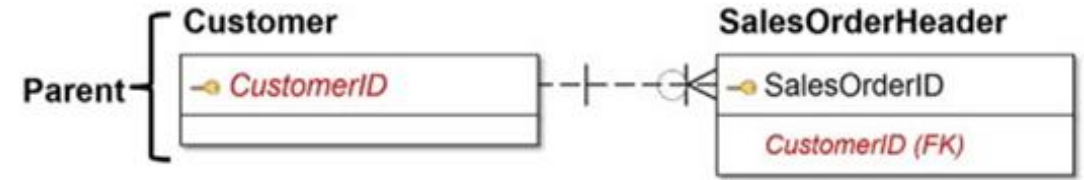
CustomerID	
13456	

BookID	
13456i00145	

Cuando se usan claves primarias compuestas en las entidades hijas que contienen la clave primaria de la entidad padres.

Modelo ER

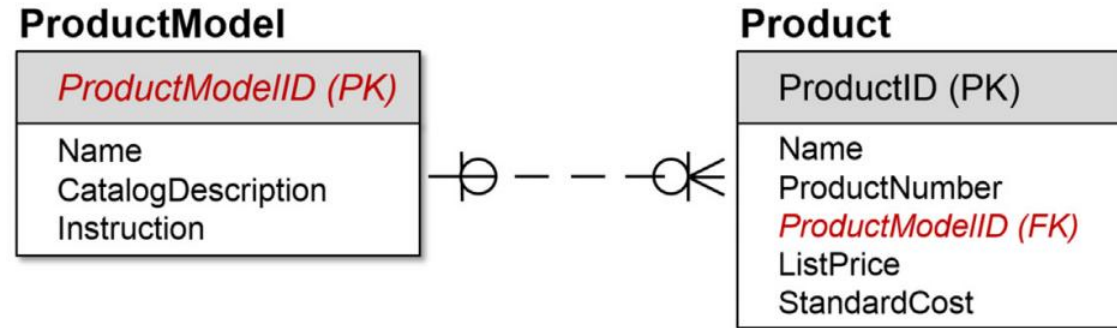
Una **relación obligatoria no identificativa** ocurre cuando una entidad (**hija**) tiene una **clave foránea (FK)** que referencia a la clave primaria (**PK**) de otra entidad (**padre**), pero **sin que esta clave foránea forme parte de la clave primaria** de la entidad hija. Además, la relación es **obligatoria**, lo que significa que cada registro en la entidad hija **debe** tener un registro asociado en la entidad padre.



- Un Pedido debe estar asociado a un Cliente, pero su ID_Pedido es su propia PK independiente. ID_Cliente es la clave foránea (FK) en Pedido, pero la clave primaria (PK) de Pedido sigue siendo ID_Pedido.
- Un pedido no puede existir sin un cliente (relación obligatoria).
- La clave foránea no forma parte de la PK de Pedido, por lo que la relación es no identificativa.

Modelo ER

Una **relación opcional no identificativa** ocurre cuando una entidad (hija) tiene una clave foránea (FK) que referencia a la clave primaria (PK) de otra entidad (padre), pero **sin que esta clave foránea forme parte de la clave primaria de la entidad hija**. Además, la relación es opcional, lo que significa que un registro en la entidad hija puede o no tener un registro asociado en la entidad padre.



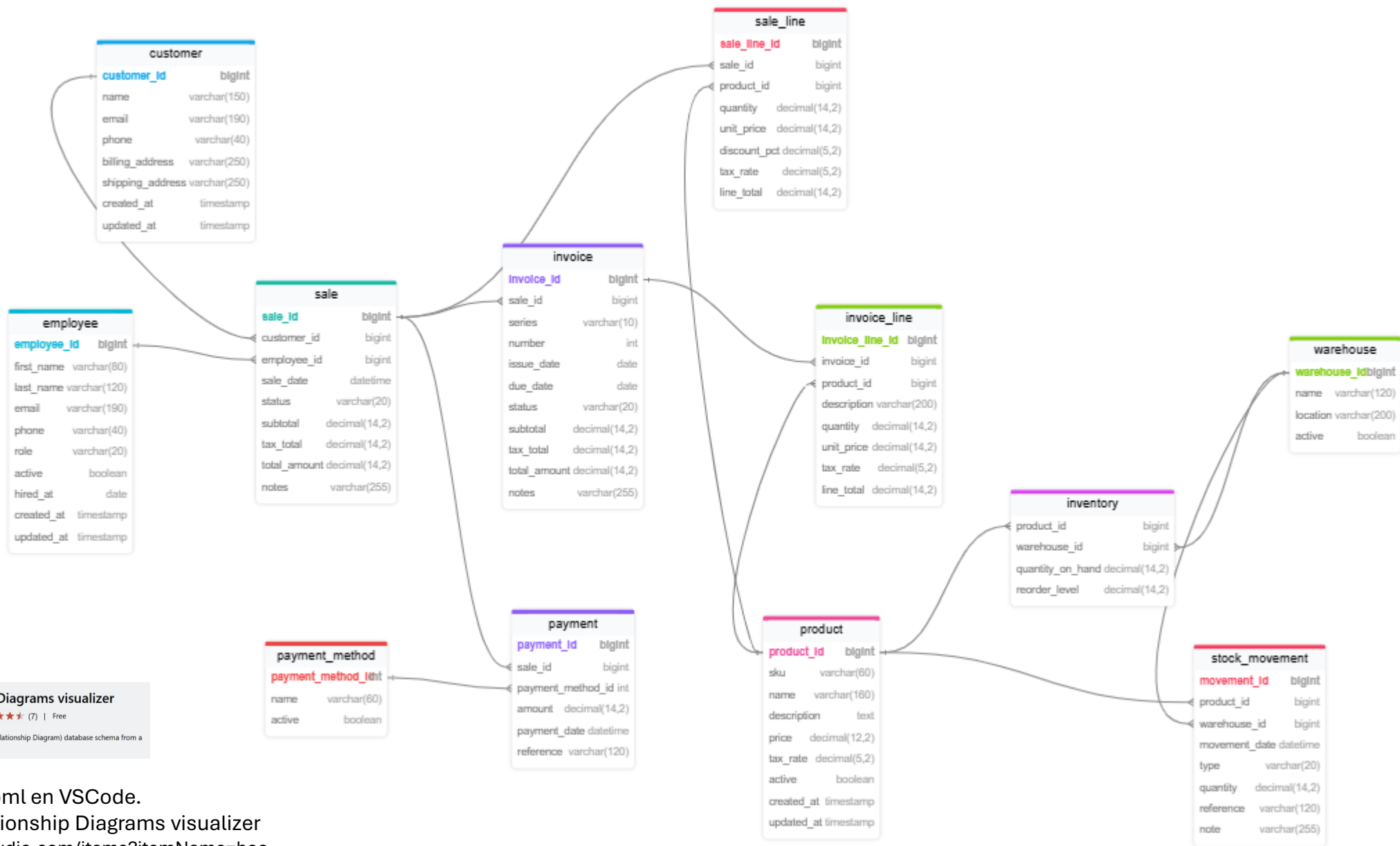
Una relación opcional no identificativa se usa cuando una entidad hija puede o no estar relacionada con una entidad padre, pero mantiene su propia clave primaria (PK).

Lenguaje para diagramas ER: un diagrama entidad-relación (ERD) no se guarda en un único lenguaje universal, sino que puede almacenarse en diferentes formatos dependiendo de la herramienta de modelado utilizada y si se busca conservar el aspecto visual o la estructura lógica.

Los métodos más comunes incluyen:

- **SQL (DDL - Data Definition Language):** Es la forma más técnica y común para diagramas físicos. Se utiliza el lenguaje de definición de datos de SQL (sentencias `CREATE TABLE`, `ALTER TABLE` para claves foráneas) para guardar la estructura de la base de datos.
- **DBML (Database Markup Language):** Un lenguaje de código abierto diseñado específicamente para definir esquemas de bases de datos de forma simple y legible. Es popular en herramientas como `dbdiagram.io`.
- **Lenguajes de Marcado/Scripting (Mermaid, PlantUML):** Permiten crear diagramas mediante texto. **Mermaid** es muy utilizado, ya que usa su propia sintaxis basada en texto para definir entidades y relaciones (ej. `erDiagram`).
- **XML (Extensible Markup Language):** Muchas herramientas de escritorio (como `Draw.io`, `Visio`) guardan los archivos en formato XML (por ejemplo, `.drawio` o `.vsdx`), que describen las posiciones de los componentes, sus formas y las conexiones entre ellos.
- **JSON/YAML:** Utilizados por herramientas modernas basadas en la web para almacenar la estructura de datos del diagrama.

En resumen, los diagramas se guardan como **código de diseño (DBML, Mermaid, SQL)** para la estructura lógica o como **archivos XML/gráficos** para la visualización.

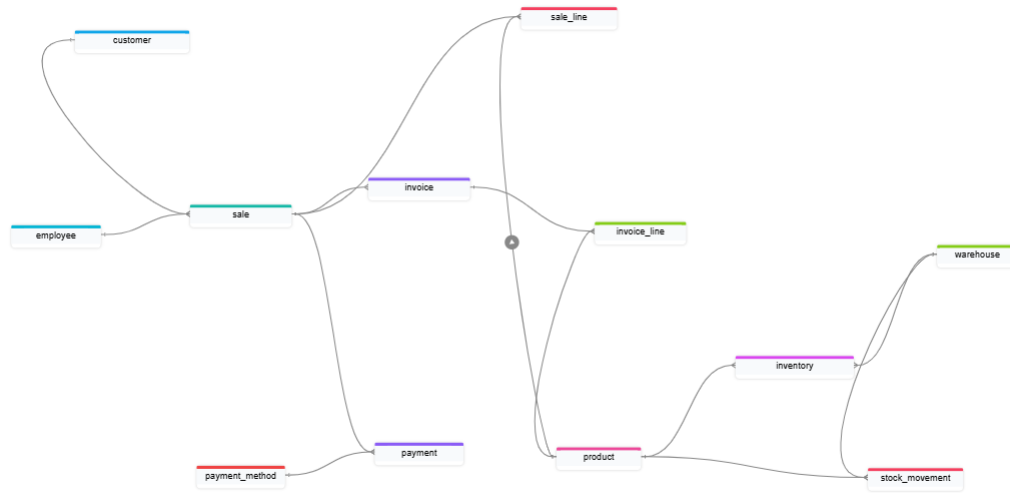




DBML Entity-Relationship Diagrams visualizer
Juste Bocovo | 26,240 installs | ★★★★★ (7) | Free

A VS Code extension for viewing the ERD (Entity Relationship Diagram) database schema from a dbml file in your VS Code Editor.

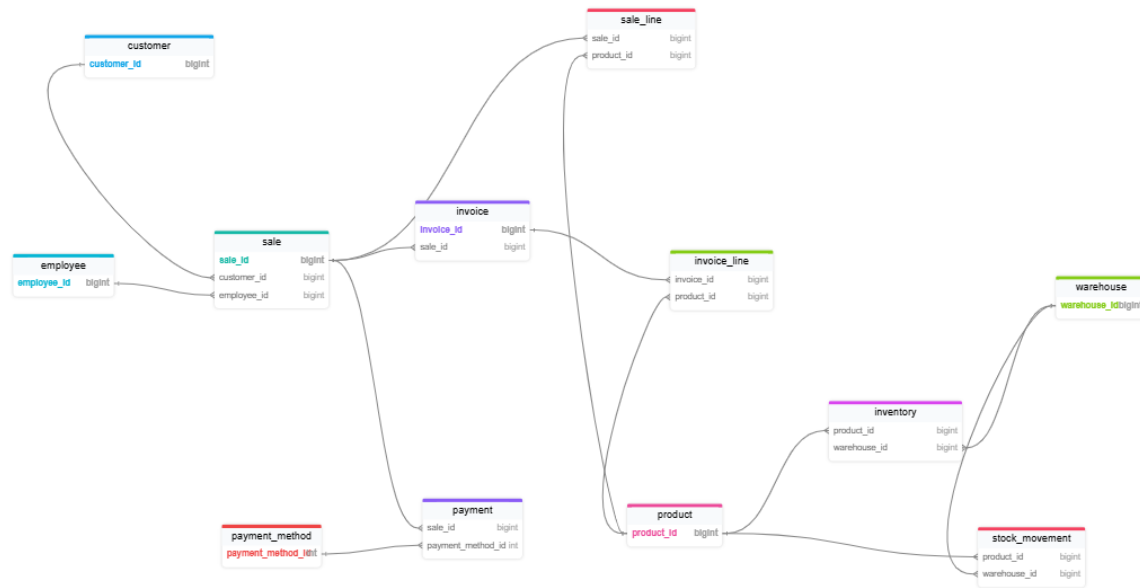
Ver ejemplo GD-Ventas.dbml en VSCode.
 Instalar DBML Entity-Relationship Diagrams visualizer
 (https://marketplace.visualstudio.com/items?itemName=bocovo.dbml-erd-visualizer&ssr=false#overview)



Ver ejemplo GD-Ventas.dbml en VSCode.

Instalar DBML Entity-Relationship Diagrams visualizer
(<https://marketplace.visualstudio.com/items?itemName=bocovo.dbml-erd-visualizer&ssr=false#overview>)

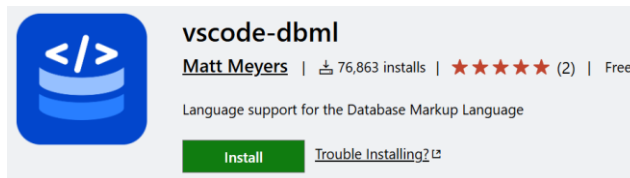
Modelo ER



Convertir a SQL:

- Abre la paleta de comandos (Ctrl+Shift+P o Cmd+Shift+P en Mac).
- Escribe "DBML: To SQL" y selecciónalo.
- Elige el dialecto de tu base de datos (PostgreSQL, MySQL o SQL Server).
- Elige el nombre y ubicación del archivo .sql de salida.

Resultado: se generará un nuevo archivo .sql con los comandos CREATE TABLE necesarios.



Ver ejemplo GD-Ventas.dbml en VSCode.
Instalar vscode-dbml
(<https://marketplace.visualstudio.com/items?itemName=matt-meyers.vscode-dbml>)

```

DiagER > GD-Ventas.sql
1 CREATE TABLE `customer` (
2   `customer_id` bigint PRIMARY KEY AUTO_INCREMENT,
3   `name` varchar(150),
4   `email` varchar(190) UNIQUE,
5   `phone` varchar(40),
6   `billing_address` varchar(250),
7   `shipping_address` varchar(250),
8   `created_at` timestamp,
9   `updated_at` timestamp
10 );
11
12 CREATE TABLE `employee` (
13   `employee_id` bigint PRIMARY KEY AUTO_INCREMENT,
14   `first_name` varchar(80),
15   `last_name` varchar(120),
16   `email` varchar(190) UNIQUE,
17   `phone` varchar(40),
18   `role` varchar(20),
19   `active` boolean,
20   `hired_at` date,
21   `created_at` timestamp,
22   `updated_at` timestamp
23 );
24
25 CREATE TABLE `product` (
26   `product_id` bigint PRIMARY KEY AUTO_INCREMENT,
27   `sku` varchar(60) UNIQUE,
28   `name` varchar(160),
29   `description` text,
30   `price` decimal(12,2),
31   `tax_rate` decimal(5,2),
32   `active` boolean,
33   `created_at` timestamp,
34   `updated_at` timestamp
35 );
36
37 CREATE TABLE `warehouse` (
38   `warehouse_id` bigint PRIMARY KEY AUTO_INCREMENT,
    
```

Integridad referencial

La **Integridad Referencial (RI)** es un principio fundamental en bases de datos relacionales que garantiza la coherencia y validez de las relaciones entre las tablas.

Se asegura de que las **claves foráneas (FK - Foreign Keys)** en una tabla siempre correspondan a un valor existente en la **clave primaria (PK - Primary Key)** de otra tabla.

Con esto se intenta:

- Evitar **referencias huérfanas**, es decir, registros con claves foráneas que apuntan a valores inexistentes.
- Mantener la **coherencia de los datos** al evitar eliminaciones o modificaciones que rompan la relación entre tablas.

ID_Cliente (PK)	Nombre	Email
1	Ana	ana@email.com
2	Luis	luis@email.com
3	María	maria@email.com

ID_Pedido	ID_Cliente (FK)	Fecha	Total
1001	1	2024-02-01	150€
1002	3	2024-02-02	200€

En Pedido, el campo ID_Cliente es una clave foránea (FK) que referencia ID_Cliente en Cliente. No se puede insertar un ID_Cliente en Pedido que no exista en Cliente. Si intentamos insertar en Pedido un registro con ID_Cliente = 5, fallará porque no existe un cliente con ID 5 en la tabla Cliente.

La **Integridad Referencial (RI)** es un principio fundamental en bases de datos relacionales que garantiza la coherencia y validez de las relaciones entre las tablas.

ID_Cliente (PK)	Nombre	Email
1	Ana	ana@email.com
2	Luis	luis@email.com
3	María	maria@email.com

ID_Pedido	ID_Cliente (FK)	Fecha	Total
1001	1	2024-02-01	150€
1002	3	2024-02-02	200€

Cuando una clave foránea hace referencia a una clave primaria, pueden definirse restricciones en caso de eliminación o actualización del valor referenciado.

- **Restricción "CASCADE" (Eliminación en cascada).** Si se elimina un cliente, se eliminan automáticamente sus pedidos.

Ejemplo: Si eliminamos el cliente Ana (ID_Cliente = 1), también se eliminan todos sus pedidos en Pedido.

- **Restricción "SET NULL" (Anulación de la FK).** Si se elimina el cliente, los pedidos que lo referencian se mantienen, pero el ID_Cliente se establece en NULL.

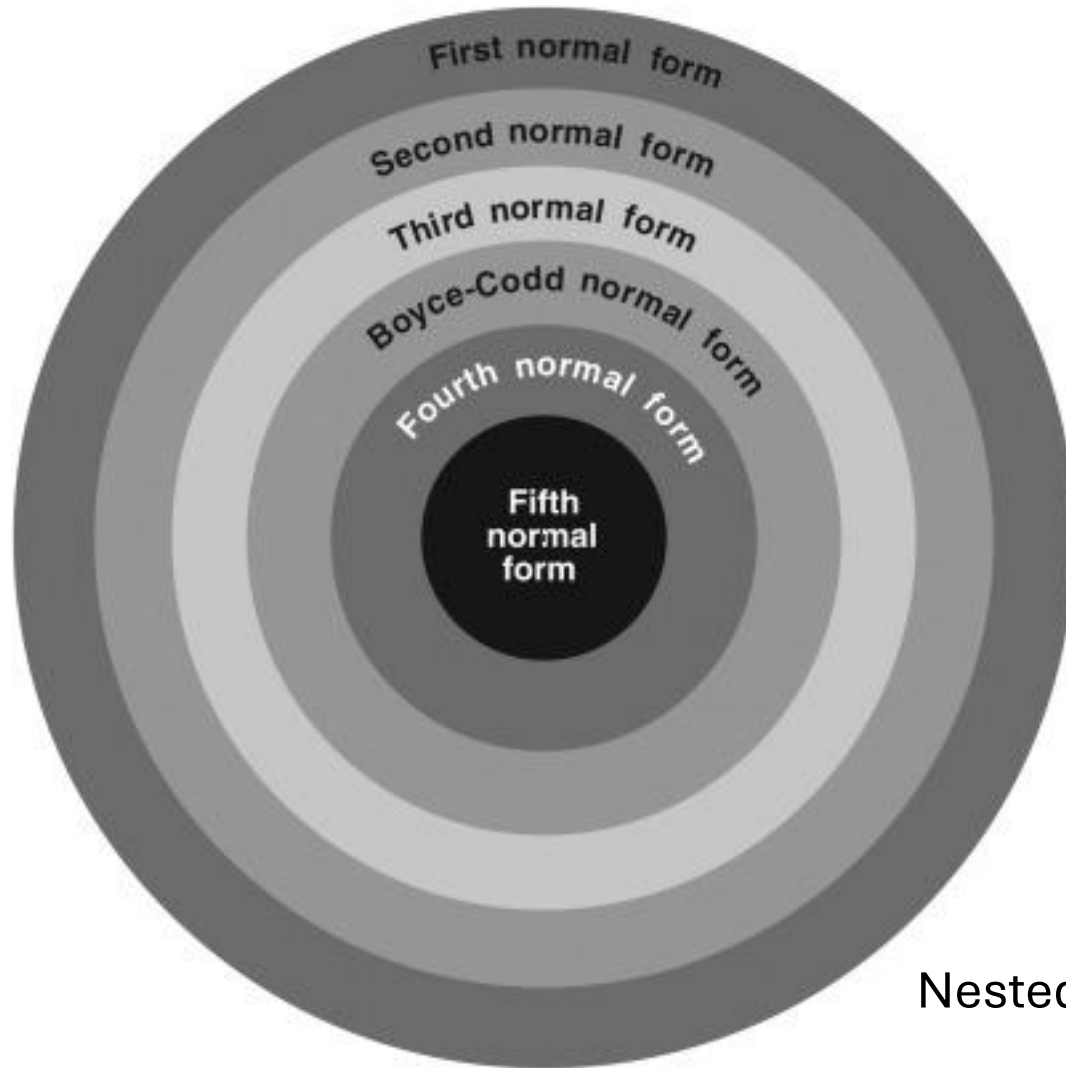
Ejemplo: Si eliminamos Luis (ID_Cliente = 2), sus pedidos seguirán existiendo, pero su ID_Cliente en Pedido será NULL.

- **Restricción "RESTRICT" o "NO ACTION" (Bloqueo de eliminación).** No permite eliminar un cliente si tiene pedidos asociados.

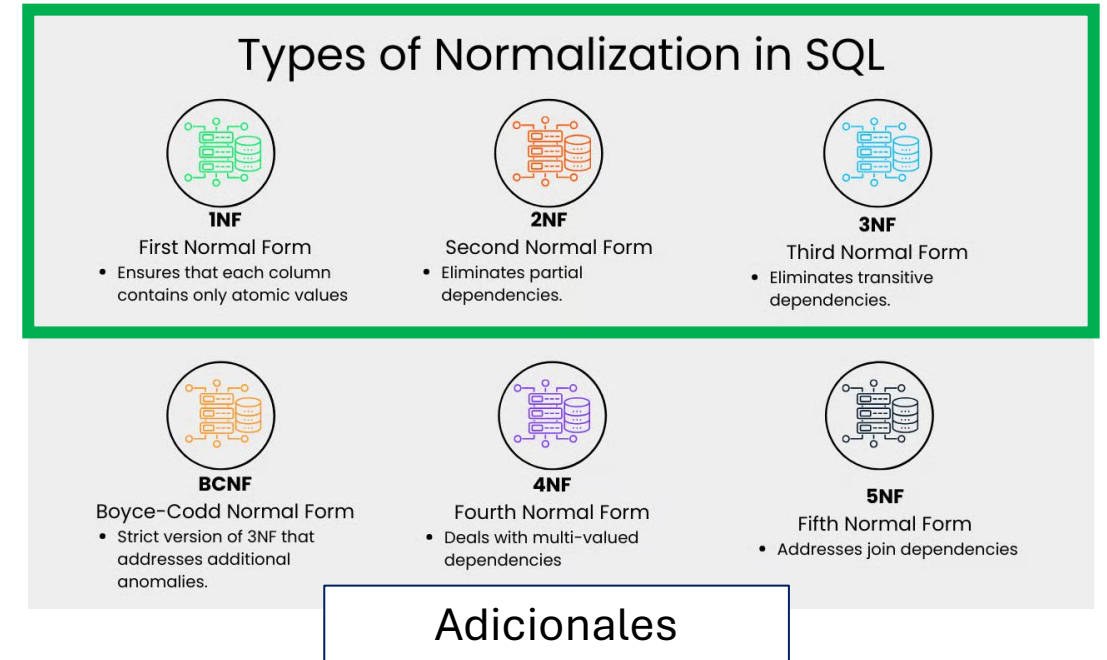
Ejemplo: Si intentamos eliminar a María (ID_Cliente = 3) mientras tiene pedidos en Pedido, la operación fallará.

Normalización de BBDD: Formas normales

Conjunto de reglas aplicadas, de forma progresiva, a las tablas de una base de datos para minimizar redundancias, reducir anomalías de actualización y asegurar la integridad de los datos.



Nested normal forms



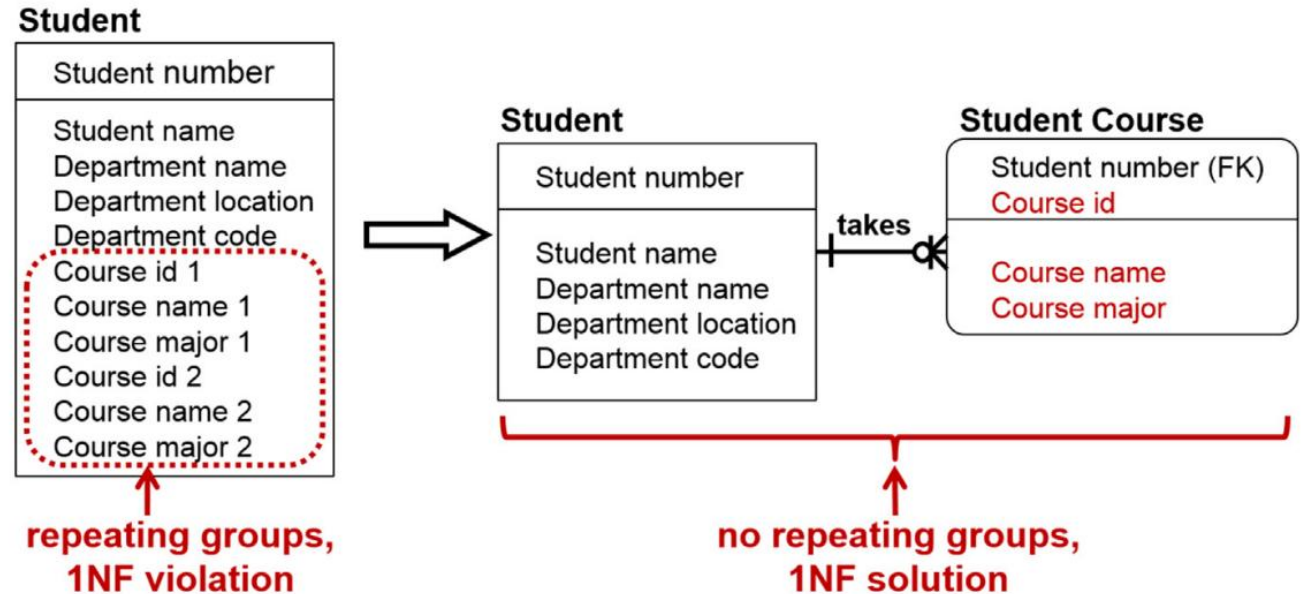
Primera Forma Normal (1NF) – Eliminación de Grupos Repetidos

Objetivo: Garantizar que los datos sean atómicos y organizar los atributos en tablas estructuradas correctamente.

Acciones a realizar:

- Cada celda debe contener un solo valor atómico (sin listas o conjuntos de valores).
- Cada tabla debe representar una única entidad sin datos repetidos dentro de una misma fila.
- Se debe definir una clave primaria (PK) para cada tabla.

Corrección si no se cumple: si una tabla no tiene una clave primaria o los datos aún presentan valores repetidos en una celda, es necesario dividirla en tablas más específicas hasta cumplir con la 1NF.



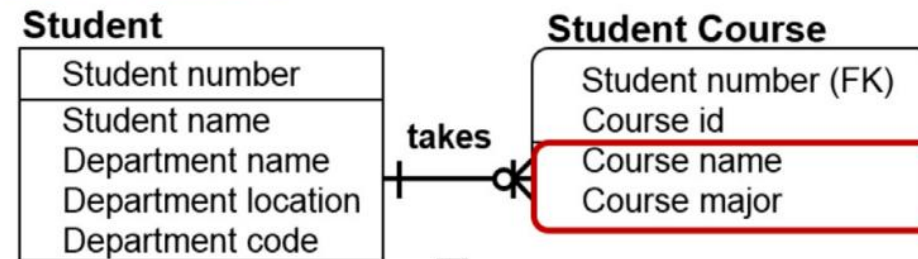
Segunda Forma Normal (2NF) – Eliminación de Datos Redundantes

Objetivo: Asegurar que cada atributo de la tabla depende completamente de la clave primaria y no solo de una parte de ella.

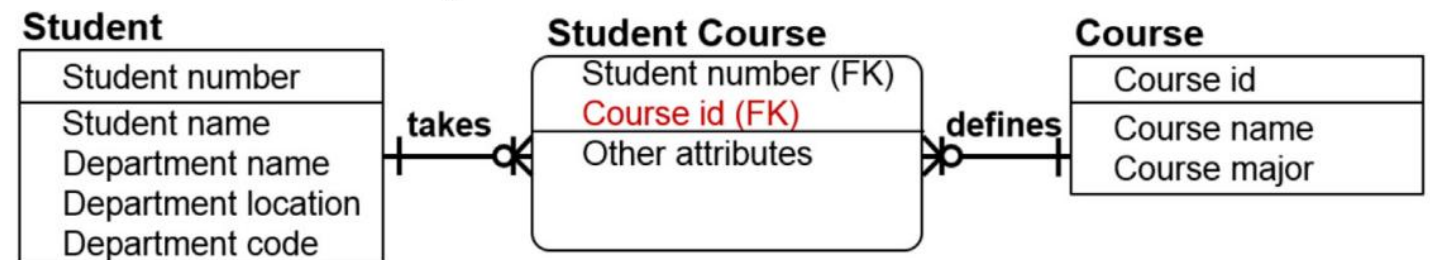
Acciones a realizar:

- Cumplir con **1NF**.
- Si la clave primaria es compuesta (formada por varios atributos), verificar que **cada atributo dependa completamente** de la clave primaria completa y no solo de una parte de ella.
- Si un atributo depende de solo una parte de la clave primaria, moverlo a una tabla separada.

2NF violation:



2NF solution:



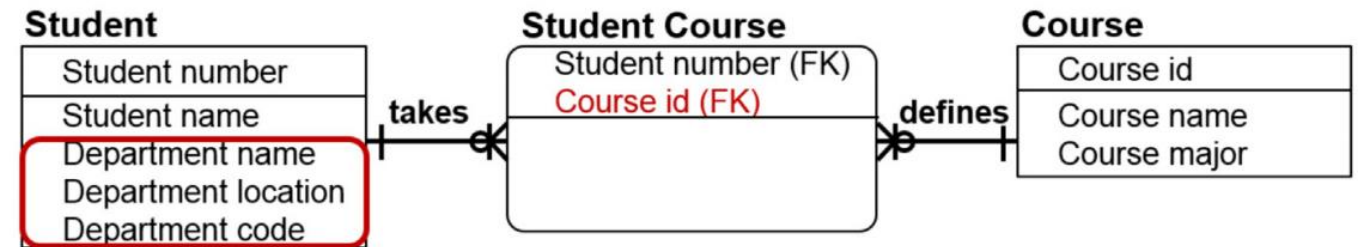
Tercera Forma Normal (3NF) – Eliminación de Dependencias Transitivas

Objetivo: Asegurar que cada atributo no clave dependa únicamente de la clave primaria y no de otros atributos no clave.

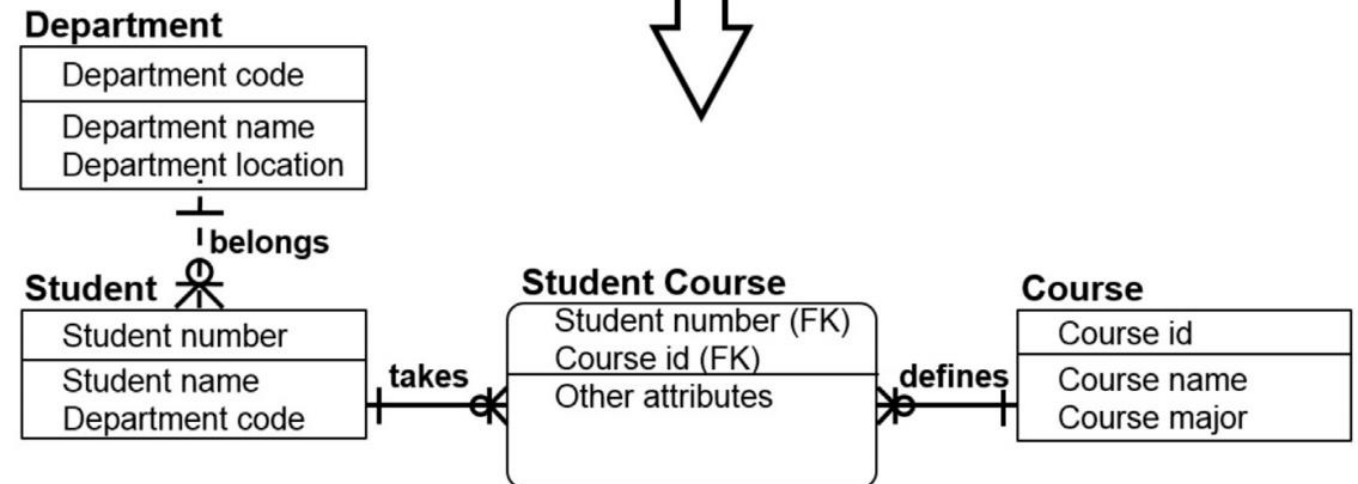
Acciones a realizar:

- Cumplir con **2NF**.
- Si un atributo no clave depende de otro atributo no clave en lugar de la clave primaria, trasladarlo a una nueva tabla.

3NF violation:



3NF solution:



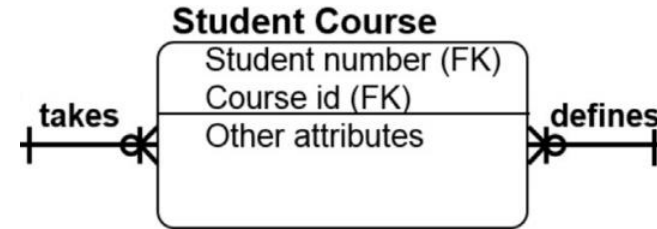
Forma Normal de Boyce-Codd (BCNF, 3.5FN) – Elimina anomalías por múltiples claves candidatas

Objetivo: Resolver anomalías causada por múltiples claves candidatas compuestas que se superponen.

Las dependencias funcionales (DF) definen las relaciones entre atributos dentro de una base de datos relacional. Un DF establece que el valor de una columna determina el valor de otra columna relacionada.

Garantiza que cada determinante (conjunto de atributos que identifican de forma única una fila) de una tabla sea una clave candidata (conjunto mínimo de atributos que identifican de forma única una fila).

Los determinantes deben poder servir como claves primarias.



¿PK de Student Course?

Enfoque 1: hacer que el conjunto de atributos combinados sea la clave primaria.

Enfoque 2: descomponer la tabla.

Enfoque 3: crear una clave primaria independiente

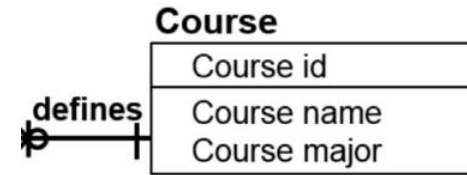
Enfoque 4: sustentar la anomalía.

Cuarta Forma Normal (4NF) – Eliminación de Dependencias Multivaluadas

Objetivo: Minimizar los valores múltiples en los atributos.

Existe una relación multivaluada cuando un atributo puede tener varios atributos dependientes, y estos atributos dependientes son independientes de la clave primaria.

Derivar a nuevas tablas, especialmente si son atributos reutilizables en distintas tuplas.



Add attribute: Course keywords

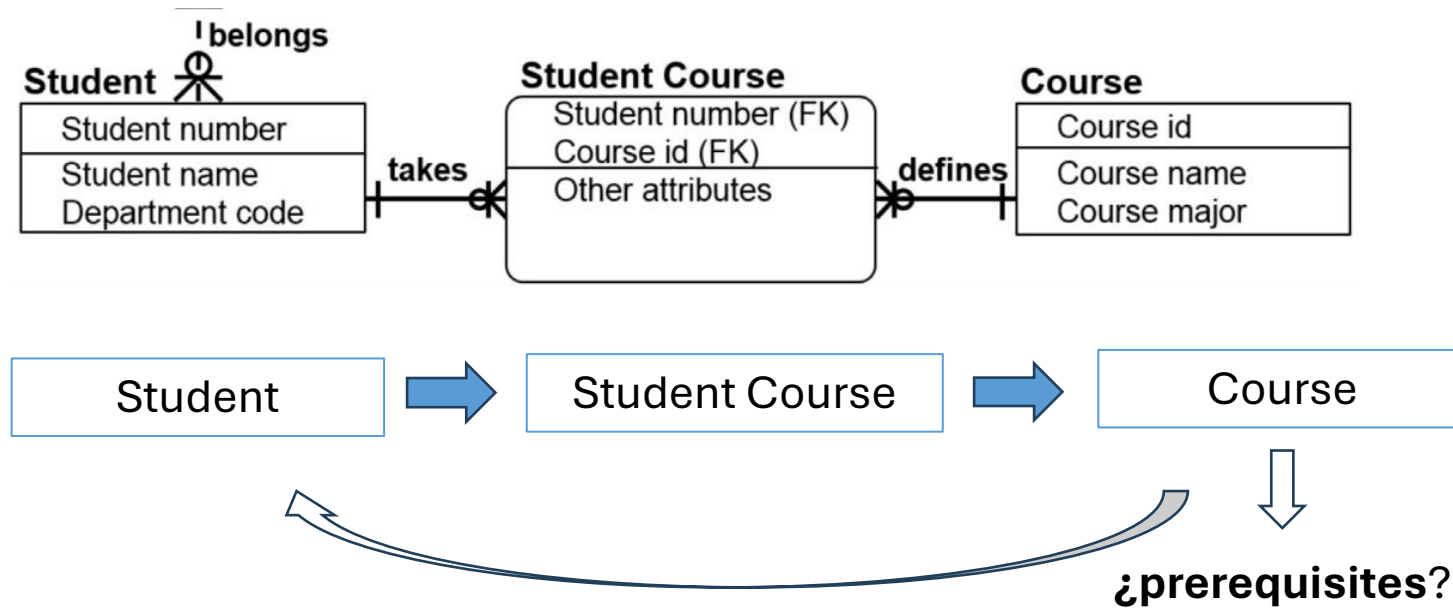
Enfoque 1: mantener una lista de keywords.

Enfoque 2: descomponer la tabla.

- Tabla de keywords por CourseId
- Tablas CourseId-KwId y Keywords

Quinta Forma Normal (5NF) – Solucionar Dependencias de Unión

Objetivo: Garantizar que al unir tablas no se pueda obtener información adicional que no estuviera ya disponible en las tablas separadas.



Causas de la necesidad de normalización

- **Anomalías de inserción, borrado y actualización:** Cualquier forma de cambio en una tabla puede provocar errores o incoherencias en otras tablas si no se maneja con cuidado. Puede provocar una pérdida involuntaria de datos.
- **Dificultad para gestionar las relaciones:** Resulta más difícil mantener relaciones complejas en una estructura no normalizada.
- **Dependencias parciales:** pueden provocar redundancia de datos y anomalías de actualización.
- **Dependencias transitivas:** pueden provocar anomalías de datos.

Dependencia Parcial: en claves primarias compuestas.

Ej. Student_Id, Course_Id: si Student_Name solo depende de Student_Id viola 2FN -> separar atributos dependientes en una nueva tabla.

Dependencia Transitiva: cuando $A \rightarrow B$ y $B \rightarrow C$, lo que implica $A \rightarrow C$.

Ej. City depende de Zip_Code y Zip_Code depende de Student: viola 3FN -> separar atributos dependientes en una nueva tabla.

Si una tabla no está correctamente normalizada y tiene redundancia de datos, no sólo ocupará espacio extra de almacenamiento de datos, sino que también dificultará el manejo y la actualización de la base de datos.

Desnormalización

Es una técnica de optimización que introduce estratégicamente redundancia en una base de datos relacional, combinando tablas o agregando datos duplicados para mejorar la velocidad de lectura y el rendimiento de las consultas

- **Propósito:** Optimizar el rendimiento de las consultas, especialmente en grandes volúmenes de datos donde la normalización (3NF/BCNF) hace que las consultas sean demasiado lentas.

- **Técnicas Comunes:** Combinar tablas (denormalización), agregar columnas redundantes, crear vistas materializadas y almacenar valores calculados.

- **Ventajas:** Consultas más rápidas, menor necesidad de JOINS complejos, ideal para data warehouses.

- **Desventajas:**

- Mayor espacio de almacenamiento
- Riesgo de inconsistencia de datos al tener la misma información en múltiples lugares
- Actualizaciones más complejas y lentas

La desnormalización **no** significa saltarse la normalización; al contrario, generalmente se aplica sobre un diseño previamente normalizado para ajustar el rendimiento a requisitos específicos.

Desnormalización: algunos casos de uso

Paneles de control e informes de BI (OLAP/análisis)

- Precalcular los agregados diarios/mensuales
- Mantener vistas de agrupaciones costosas
- Almacenar tablas de hechos desnormalizadas para segmentos comunes

Comercio electrónico/catálogo y páginas de búsqueda/listado:

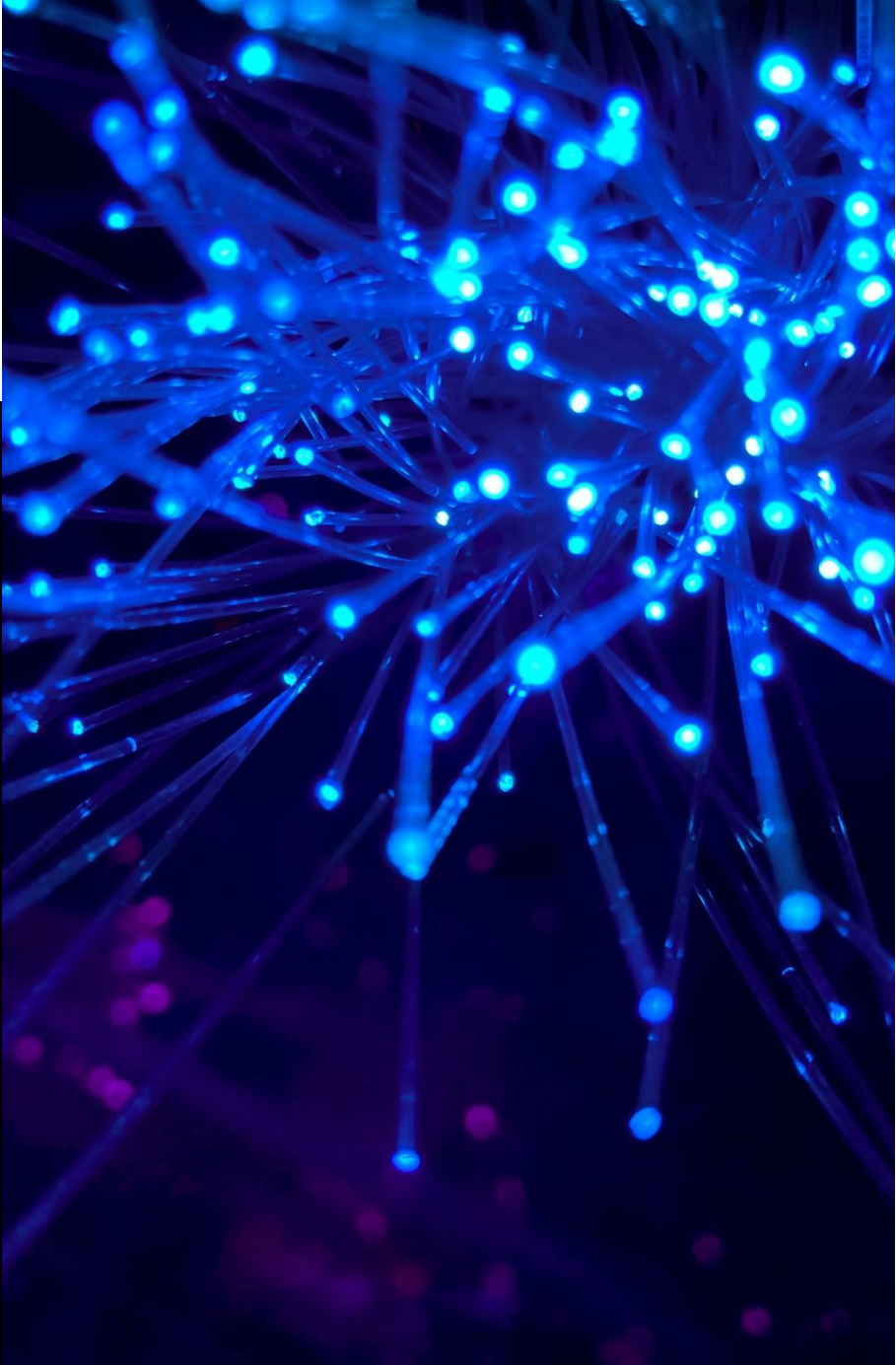
- Preparar datos para listas y filtros rápidos: duplicar nombre_categoría, nombre_marca, precio_con_impuestos o preparar una proyección de productos.

Feed de actividad y contadores:

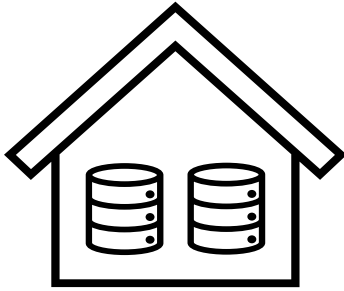
- Mantener like_count, follower_count, o latest_comment_at como atributos derivados en lugar de recalcularlos.

Análisis de eventos/registros a gran escala (OLAP/nosql):

- Preparar los datos anidados y diseñar las particiones para que los escaneos sean predecibles.



Modelo de datos en DWH modernos

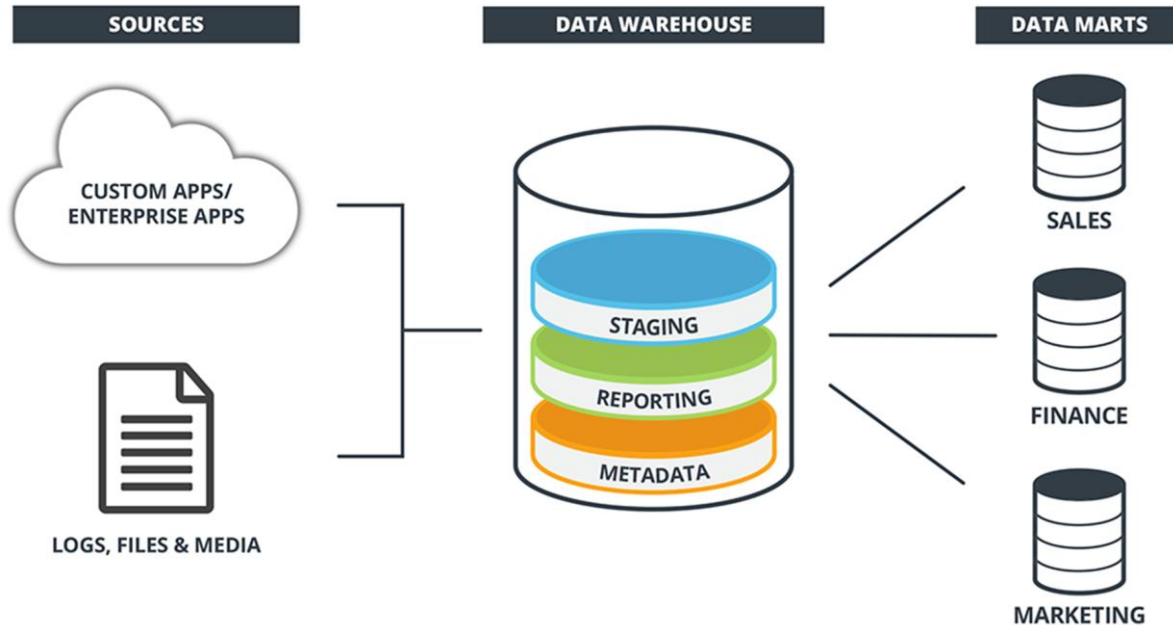


Un **Data Warehouse** es un sistema centralizado diseñado para almacenar, organizar y gestionar datos históricos provenientes de múltiples fuentes operacionales.

Estos datos están estructurados, integrados y preparados para análisis y reportes, lo que permite la toma de decisiones estratégicas en una organización.

Características clave:

- **Centralizado:** Unifica datos de múltiples sistemas operativos.
- **Histórico:** Almacena datos a lo largo del tiempo para permitir análisis de tendencias.
- **Estructurado:** Los datos se organizan en tablas y esquemas definidos, optimizados para consultas.
- **Optimizado para análisis:** Facilita el uso de herramientas de BI (*Business Intelligence*) para reportes y análisis.



Metadata es "datos sobre datos".

En el contexto de un Data Warehouse, describe las propiedades, estructuras y contexto de los datos almacenados.

Es esencial para la gestión, integración y análisis de datos.

La **etapa de reporting** en un Data Warehouse es la capa final donde los datos procesados y organizados se presentan a los usuarios para análisis, toma de decisiones y generación de reportes.

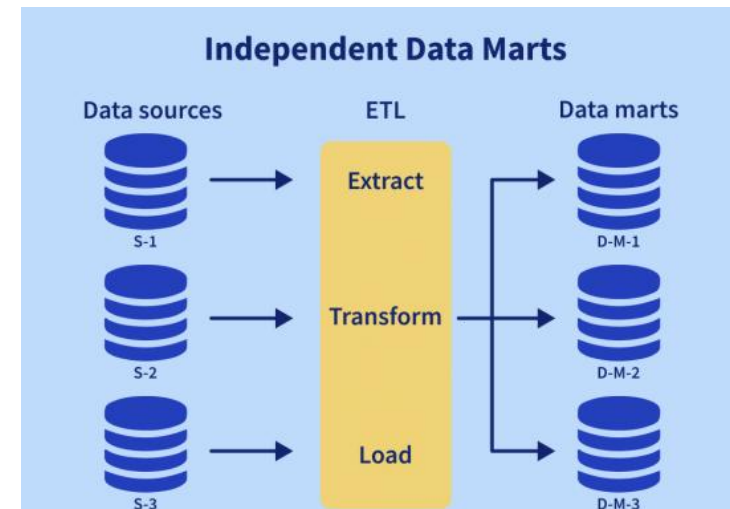
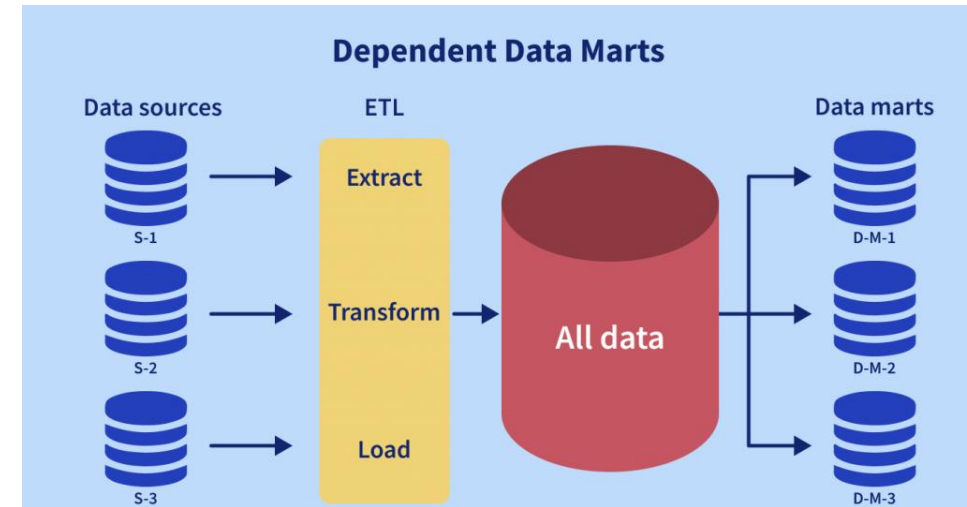
La **Staging Area** es una capa temporal que recopila, limpia y transforma datos de diversas fuentes antes de cargarlos en el Data Warehouse.

Los datos permanecen en esta etapa únicamente durante el procesamiento, pudiendo estar en su formato original. Su flexibilidad permite realizar transformaciones masivas sin afectar el almacén principal.

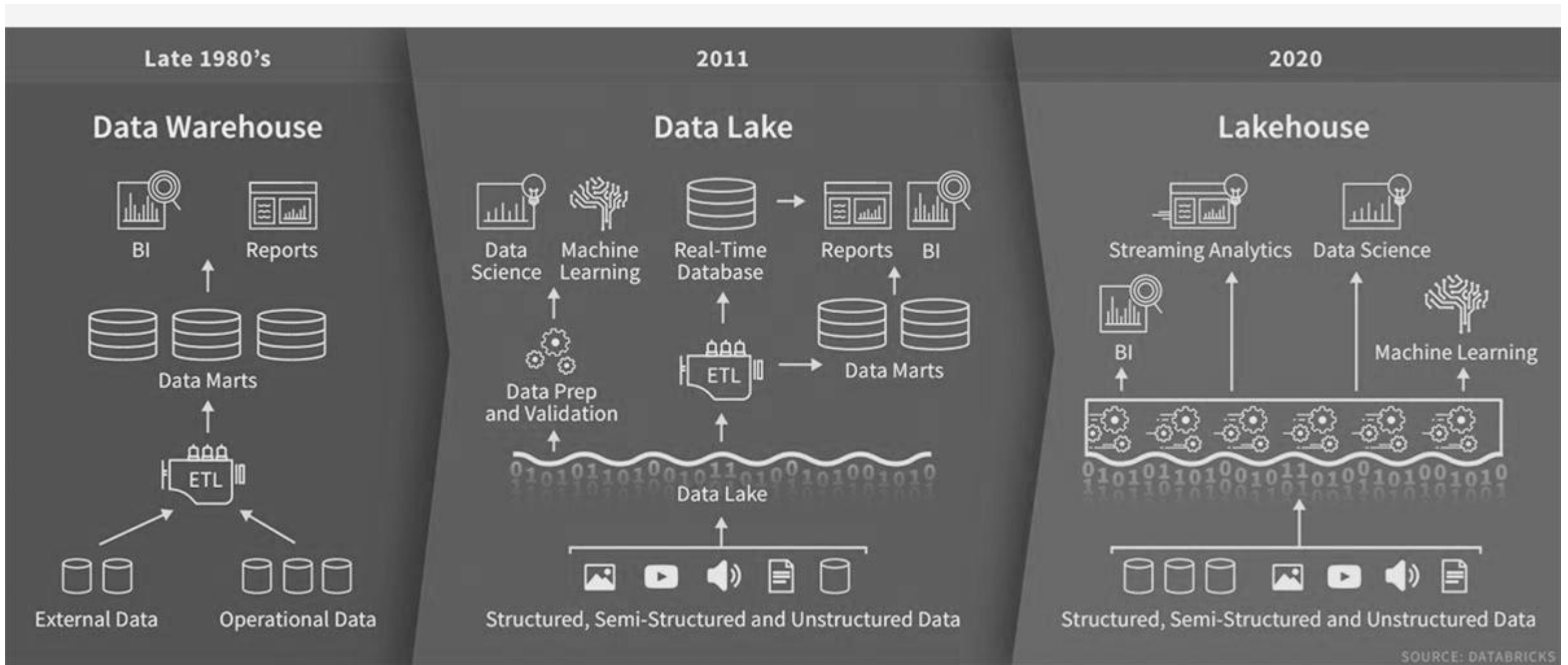
Sus funciones clave incluyen la extracción de datos de múltiples sistemas (ERP, CRM, APIs), limpieza de errores o inconsistencias, aplicación de reglas de negocio y consolidación de datos en un formato común para garantizar su consistencia y preparación para análisis.

Un **Data Mart** es un subconjunto de un **Data Warehouse** que está diseñado para satisfacer las necesidades específicas de análisis de datos de un departamento o área funcional dentro de una organización, como ventas, finanzas, marketing o recursos humanos.

Los Data Marts están diseñados para responder a necesidades analíticas específicas de un equipo o departamento. Por ejemplo, un Data Mart para el equipo de ventas podría contener datos de clientes, productos, regiones y métricas de ventas.



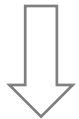
ARQUITECTURAS EN EL SIGLO XXI ... LA EVOLUCIÓN HACIA UN MODELO ESCALABLE....



DATA WAREHOUSE

[estructurado, rápido, rígido]

- procesa y limpia datos antes del almacenamiento (ETL)
- optimizado para análisis de SQL, reportes y BI



mejor opción para conservar datos grandes y sin refinar para análisis futuros, desconocidos o exploratorios.

DATA LAKE

[sin procesar, diverso, flexible]

- almacena todo tipo de datos (estructurados, semiestructurados, sin procesar) a bajo costo
- se utiliza para big data, IA y aprendizaje automático



ideales para generar informes operativos rápidos, consistentes y bien entendidos.

DATA LAKEHOUSE

[híbrido, de alto rendimiento]

- almacenamiento flexible y de bajo costo (como un lago)
- con una gestión de datos estructurados de alto rendimiento (como un almacén)



se están convirtiendo en el estándar moderno al eliminar la necesidad de administrar lagos y almacenes de datos separados

DATA WAREHOUSE

[estructurado]

- procesa y almacena
- optimiza SQL, reprocesa



LIBRO ELECTRÓNICO

**El mejor almacén es
una casa en el lago**

Unifique datos para
desbloquear información.

mejor opción
grandes y sin
futuros,
ex

DATA LAKE

[diverso, flexible]

todo tipo de datos

datos, sin
costo

big data, IA y
automático

generar informes
, consistentes y
rendidos.

DATA LAKEHOUSE

[híbrido, de alto rendimiento]

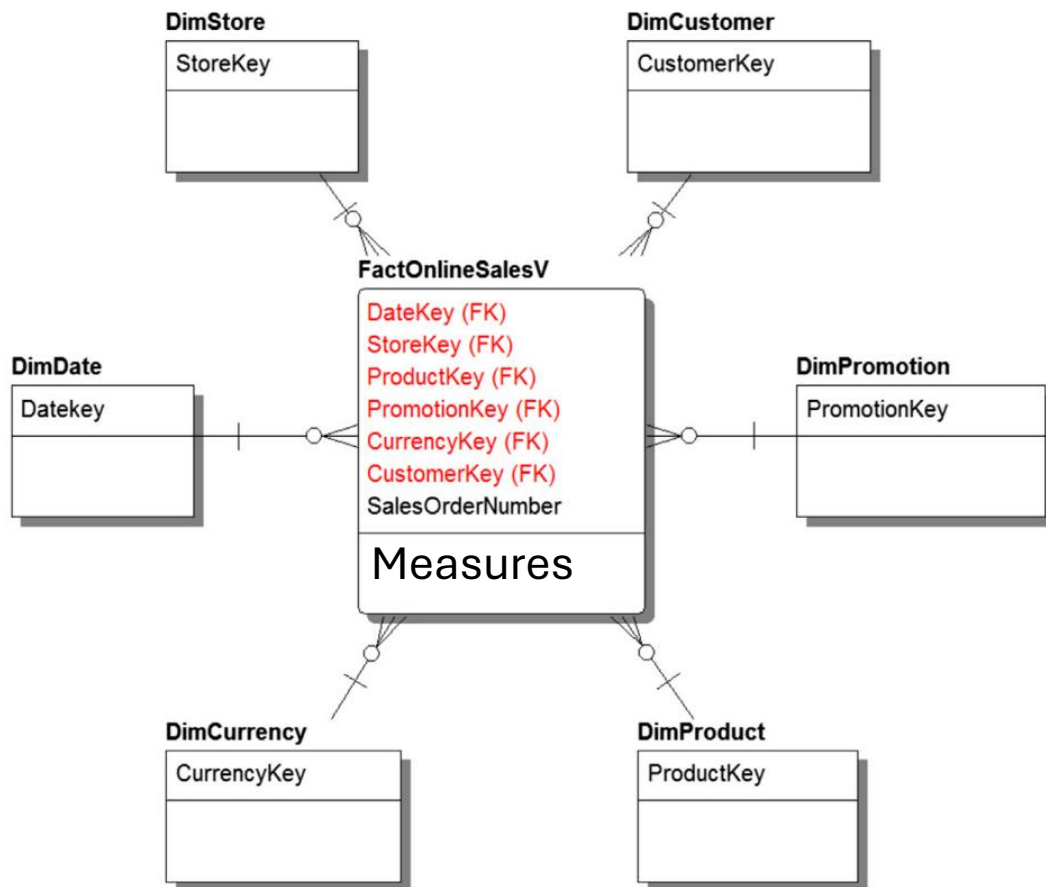
- almacenamiento flexible y de bajo costo (como un lago)
- con una gestión de datos estructurados de alto rendimiento (como un almacén)



se están convirtiendo en el
estándar moderno al eliminar la
necesidad de administrar lagos
y almacenes de datos
separados

Modelo Dimensional

El **Modelo Dimensional** es la metodología más utilizada en Data Warehouses porque permite organizar la información de manera intuitiva y optimizada para consultas analíticas.



Está compuesto por:

- **Tabla de Hechos:** Contiene datos medibles (métricas) de los eventos de negocio.
 - Cada medida tiene un nivel de granularidad que define la precisión de los datos registrados.
 - Todas las medidas de una tabla de hechos deben tener la misma granularidad, es decir, deben estar relacionadas con el mismo nivel de detalle del evento del negocio.

Ej: ingresos por ventas (SalesAmount) y cantidad de pedidos (OrderQuantity). Granularidad fina -> cada venta, gruesa -> ventas mensuales.

- **Tablas de Dimensiones:** Proporcionan contexto para analizar los datos almacenados en la tabla de hechos.
- ✓ **Facilita consultas y análisis rápidos** mediante la estructuración de datos en hechos y dimensiones.
- ✓ **Permite crear dashboards e informes eficientes** para la toma de decisiones.
- ✓ **Optimiza el rendimiento** en bases de datos analíticas, reduciendo tiempos de respuesta.

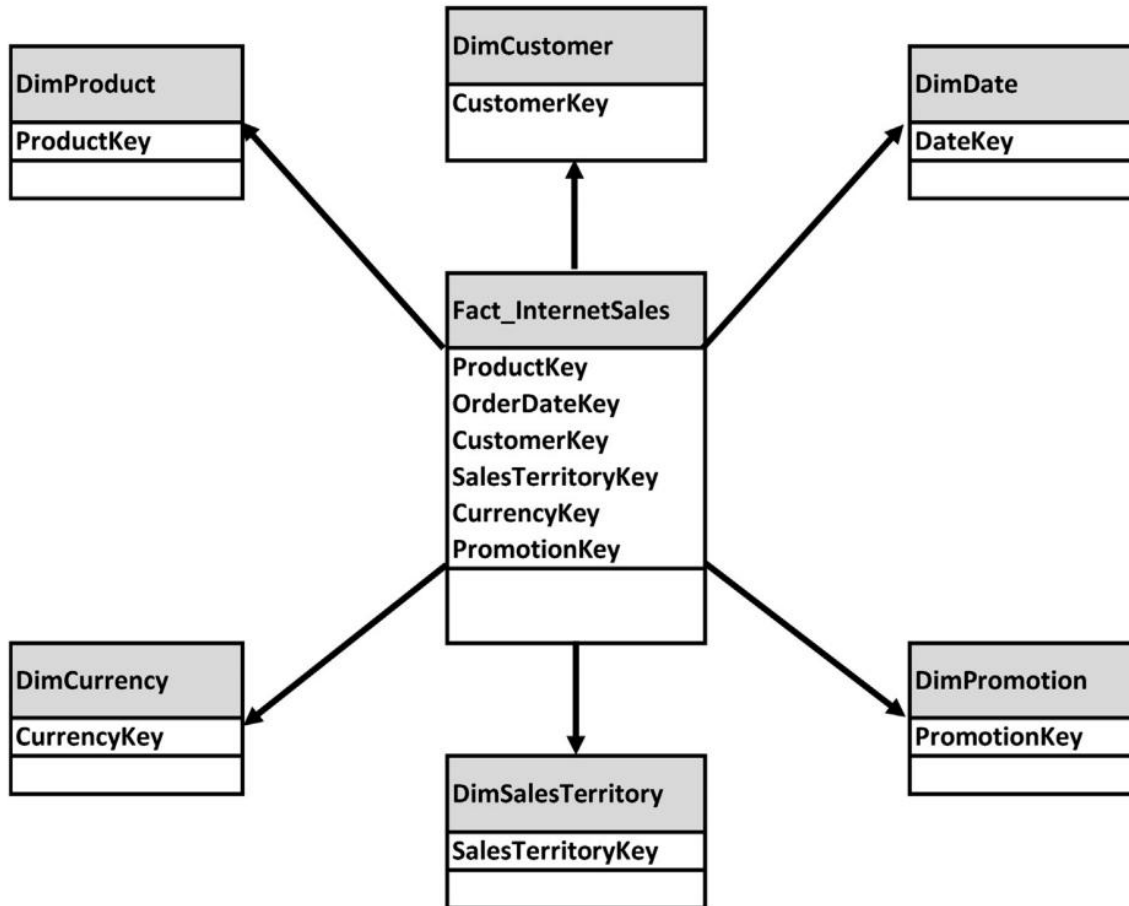
Modelo Dimensional:

- Esquema en estrella
- Esquema en copo de nieve
- Esquema constelación o galaxia

Modelo Multidimensional

OLAP (On-Line Analytical Processing)

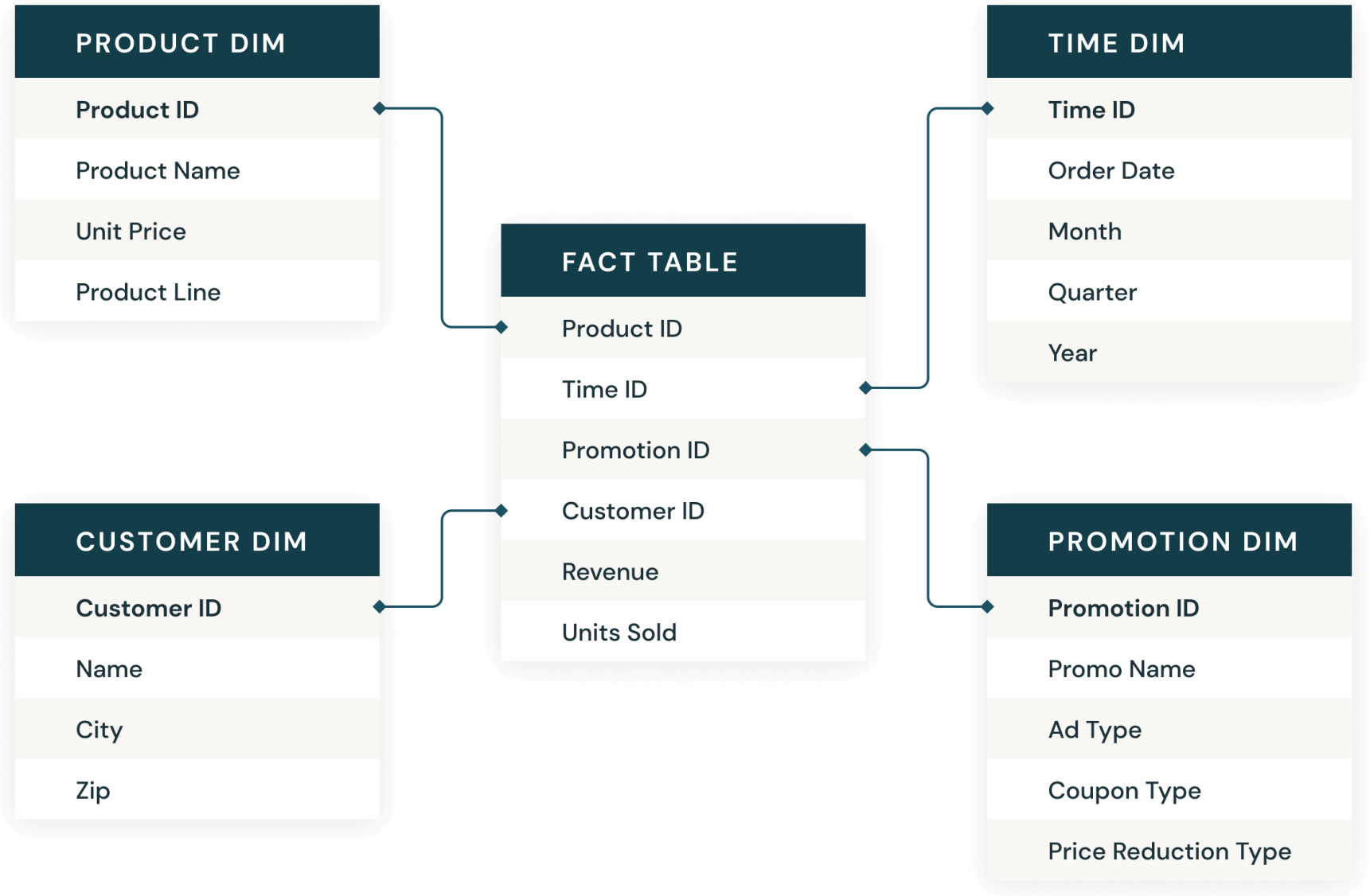
El **esquema en estrella** (Star Schema) es la estructura más simple y común. Se compone de **una tabla de hechos** conectada directamente a varias **tablas de dimensiones**, sin normalización



El esquema en estrella es una solución eficiente para consultas y análisis de datos en Data Warehouses.

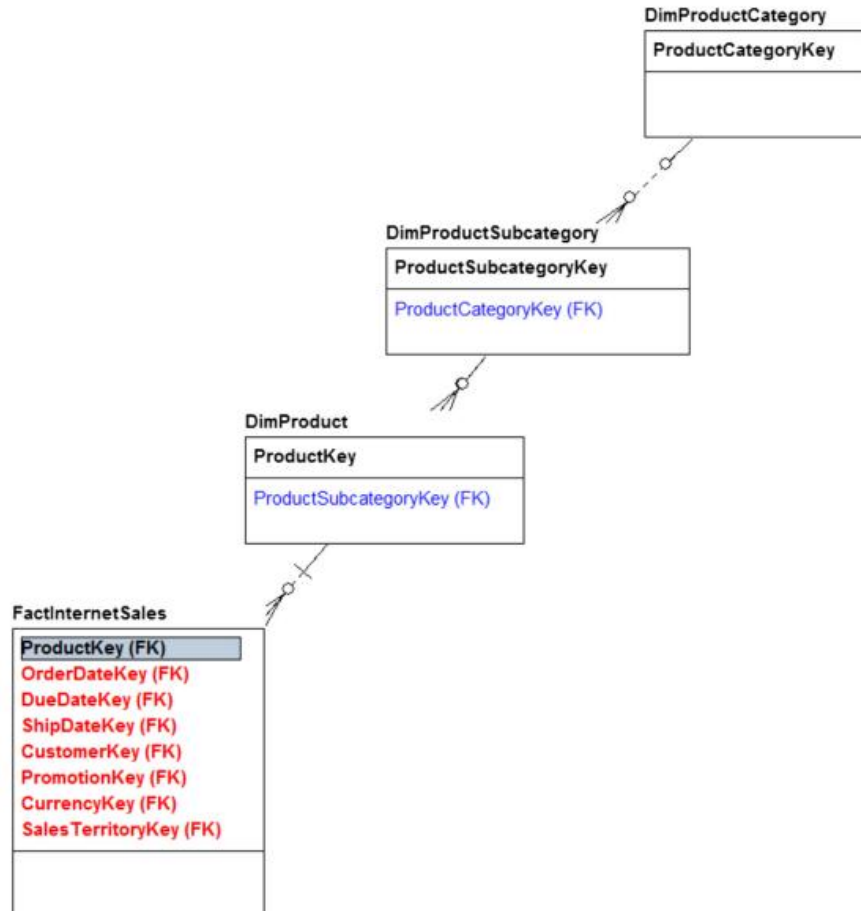
- Minimiza la complejidad para los usuarios de negocio, facilitando la creación de reportes.
- Optimiza el rendimiento al reducir la cantidad de joins entre tablas.

Star schema



El **esquema en copo de nieve** (Snowflake Schema) es una variación del esquema en estrella, en el que las dimensiones se normalizan para eliminar redundancias.

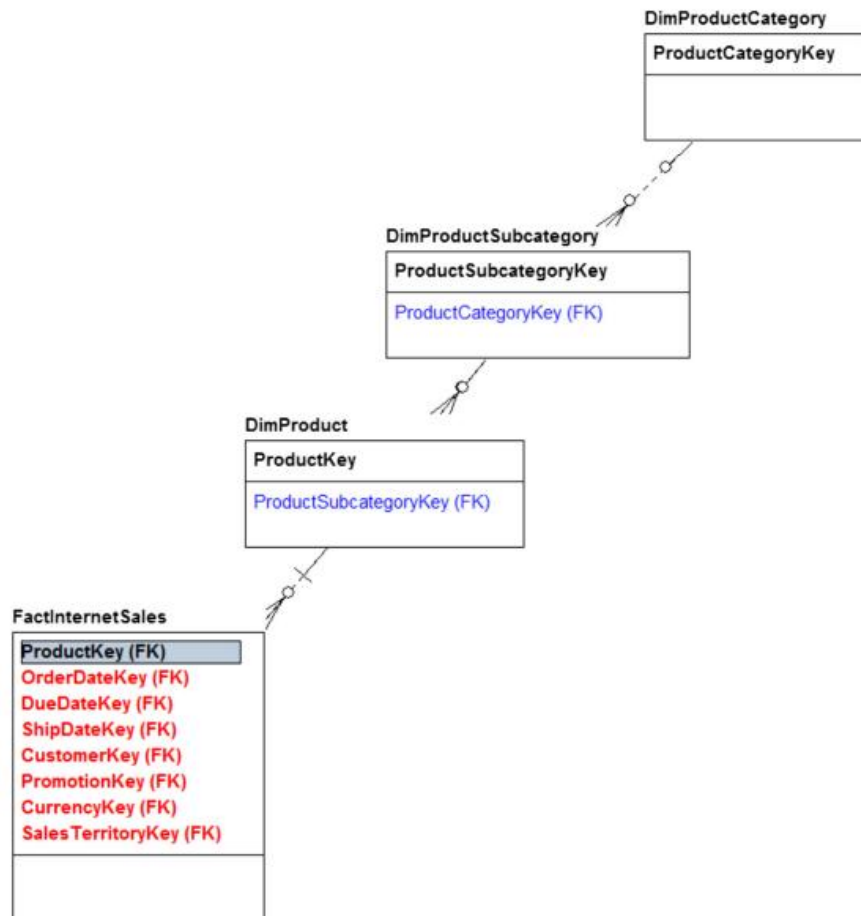
Se utiliza cuando las dimensiones tienen jerarquías naturales que pueden representarse en tablas separadas en lugar de una única tabla desnormalizada.



Es útil en sistemas donde el almacenamiento es una preocupación y las dimensiones tienen jerarquías claras.

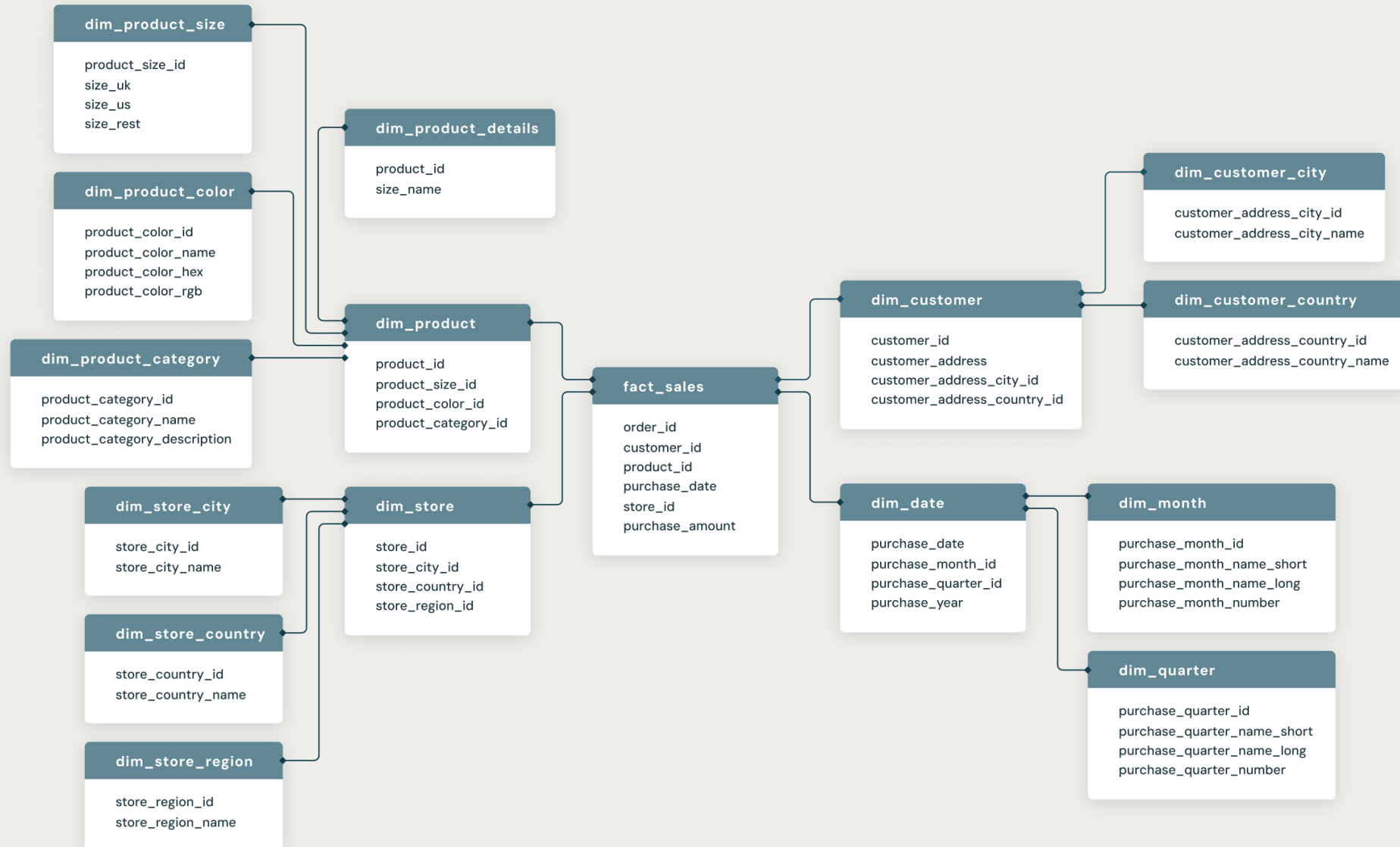
Se necesita más procesamiento debido a los múltiples joins, pero es más eficiente en la administración de datos a gran escala.

El **esquema en copo de nieve** (Snowflake Schema) es una variación del esquema en estrella, en el que las dimensiones se normalizan para eliminar redundancias.

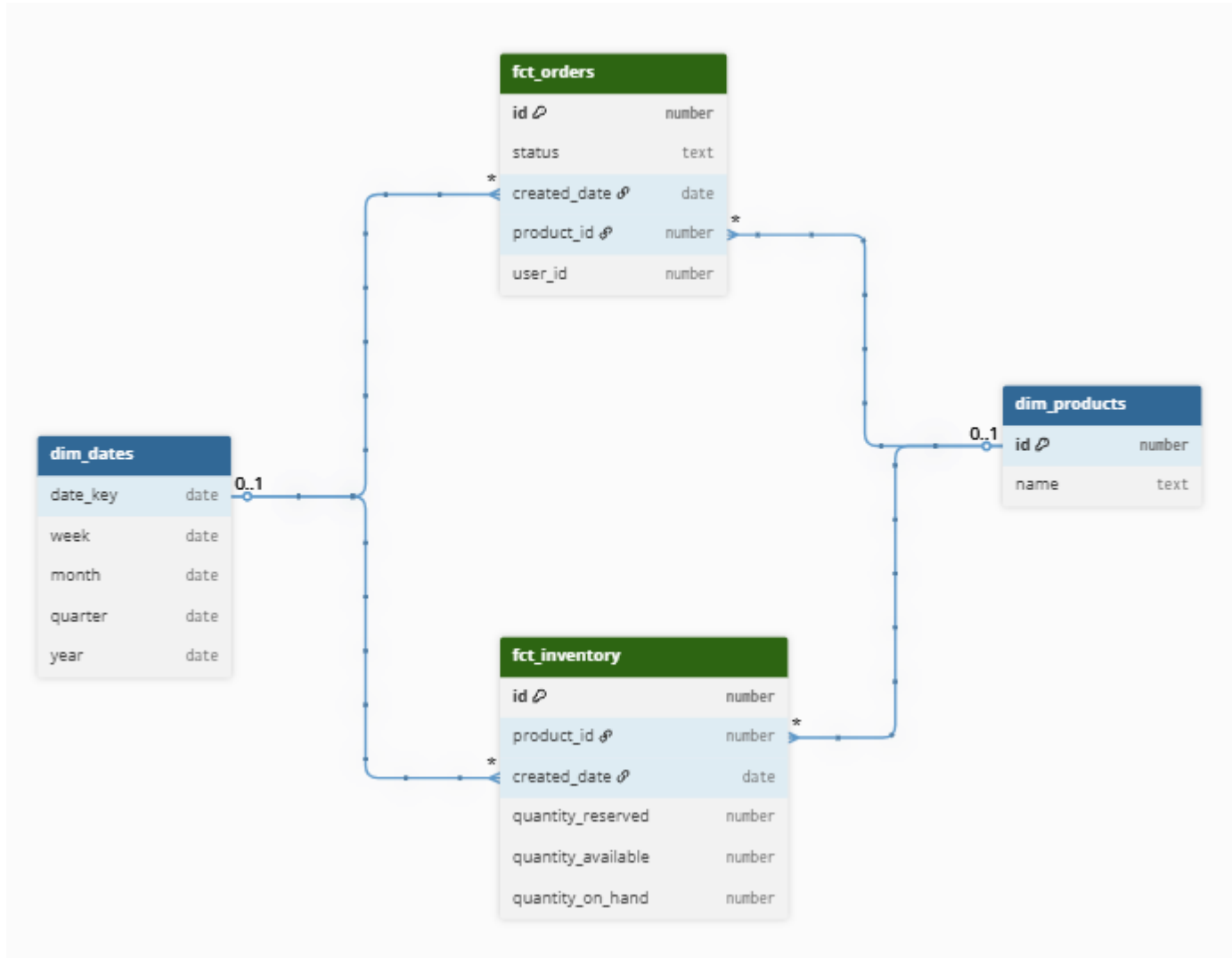


- Basado en el Esquema en Estrella, pero con Normalización en las Dimensiones
- En lugar de almacenar todas las características de una dimensión en una sola tabla (como en un Star Schema), cada nivel jerárquico se convierte en una tabla separada.
- Se crean claves primarias en cada tabla y claves foráneas que las vinculan en la estructura jerárquica.
- La tabla de hechos almacena claves foráneas (FK) que apuntan al nivel más bajo de cada jerarquía dimensional.
- Las dimensiones se descomponen en subdimensiones con relaciones padre-hijo.
- Al eliminar la duplicación de datos en las dimensiones, se reduce el almacenamiento.
- La mayor cantidad de joins entre tablas puede afectar el rendimiento de consultas.

Snowflake Schema



El **esquema constelación o galaxia** (Galaxy Schema, Fact Constellation) es una variación en la que varias tablas de hechos comparten tablas de dimensiones comunes (dimensiones conformadas).



Equivale a conectar múltiples esquemas estrella a través de dimensiones comunes.

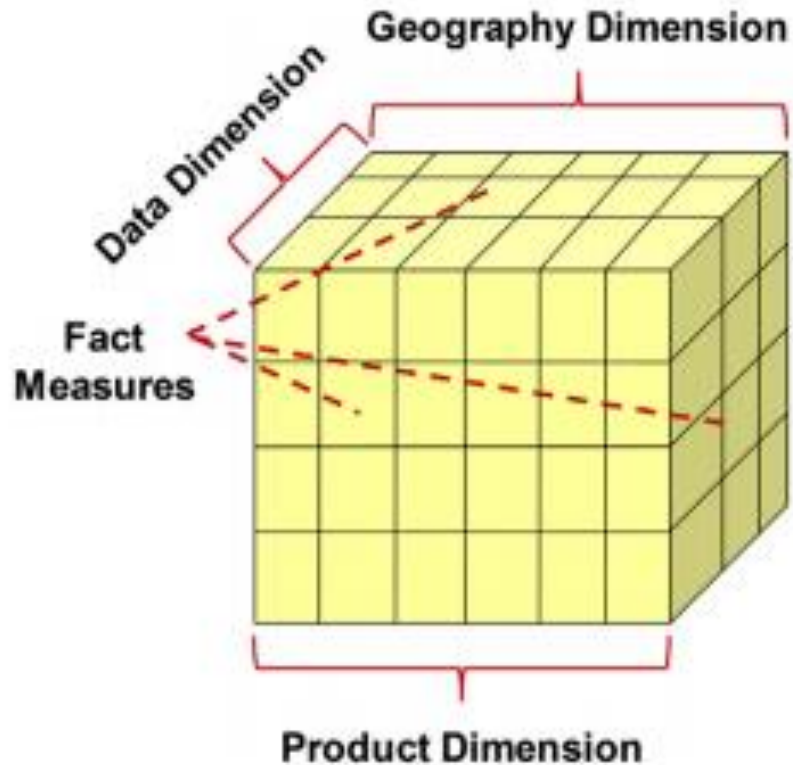
Para análisis avanzados en entornos complejos -> visión integral.

Requiere el manejo de selección de rutas, ya que puede haber múltiples rutas posibles.

Ejemplo: cantidad disponible de un producto

- dim_products->fct_inventory
- dim_products->fct_orders->dim_dates->fct_inventory

El **modelo multidimensional** es una estructura avanzada de almacenamiento de datos utilizada en **OLAP (Online Analytical Processing)** para realizar análisis de información en múltiples perspectivas. Se basa en el concepto de **cubos de datos (data cubes)**, que permiten explorar métricas mediante dimensiones interrelacionadas.



Las dimensiones representan categorías de análisis (Ej: Tiempo, Cliente, Producto, Región).

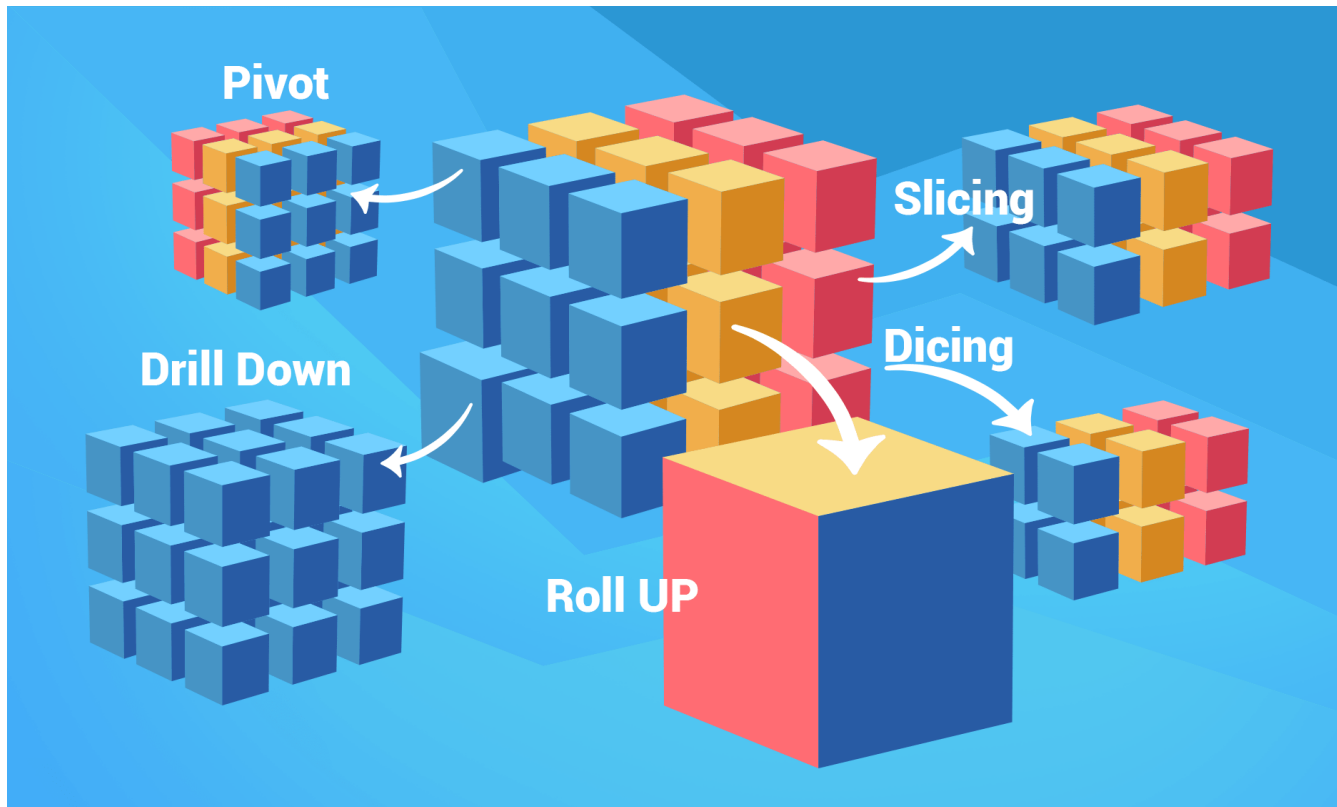
La tabla de hechos contiene valores numéricos medibles, como ventas, costos o unidades vendidas.

En análisis complejo con múltiples dimensiones.

- OLAP que requieren procesamiento de grandes volúmenes de datos.
- Cuando se buscan herramientas avanzadas como Power BI, Tableau o Cognos para análisis de datos.

Es una técnica de modelado desarrollada por Ralph Kimball para Data Warehouses.

El **modelo multidimensional** es una estructura avanzada de almacenamiento de datos utilizada en **OLAP (Online Analytical Processing)** para realizar análisis de información en múltiples perspectivas. Se basa en el concepto de **cubos de datos (data cubes)**, que permiten explorar métricas mediante dimensiones interrelacionadas.



Operaciones de manipulación:

- **Drill-Down:** Desglose de información para analizar datos más detallados.
- **Roll-Up:** Agregación de datos para ver una visión general.
- **Slice & Dice:** Filtrado y reorganización de datos para diferentes perspectivas.
- **Pivot:** Cambiar la perspectiva visual de los datos.

Las bases de datos multidimensionales se utilizan principalmente para crear aplicaciones OLAP y pueden verse como bases de datos de una sola tabla, su peculiaridad es que por cada dimensión tienen un campo (o columna), y otro campo por cada métrica o hecho.

Dimensión (Tiempo, Productos)

Jerarquía (Año->Semestre->Mes->Semana), (Categoría->Línea->Marca)

Elementos (2006, 2007, ..., S1-06, ..., Ene-06, ..., 200625....),
(Todos, LíneaA, Marca1,...)

Hechos (Ventas, Inventario, Defectos, Devoluciones)

Métricas (Devoluciones/Ventas, %Defectos)

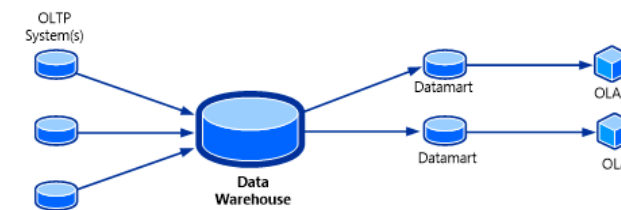
Tiempo	Productos	Ventas	Inventario	Defectos	Devoluciones	D/V	%Defectos
2006	Todos	1000	250	50	50	0,05	5

Dada esta especificación, ejemplo de tabla resultante para un hipercubo

$(d_1, d_2, d_3, \dots, f_1, f_2, f_3, \dots)$

d_i : dimensiones

f_i : hechos o métricas



Introducción a los cubos OLAP
de Service Manager para el
análisis avanzado

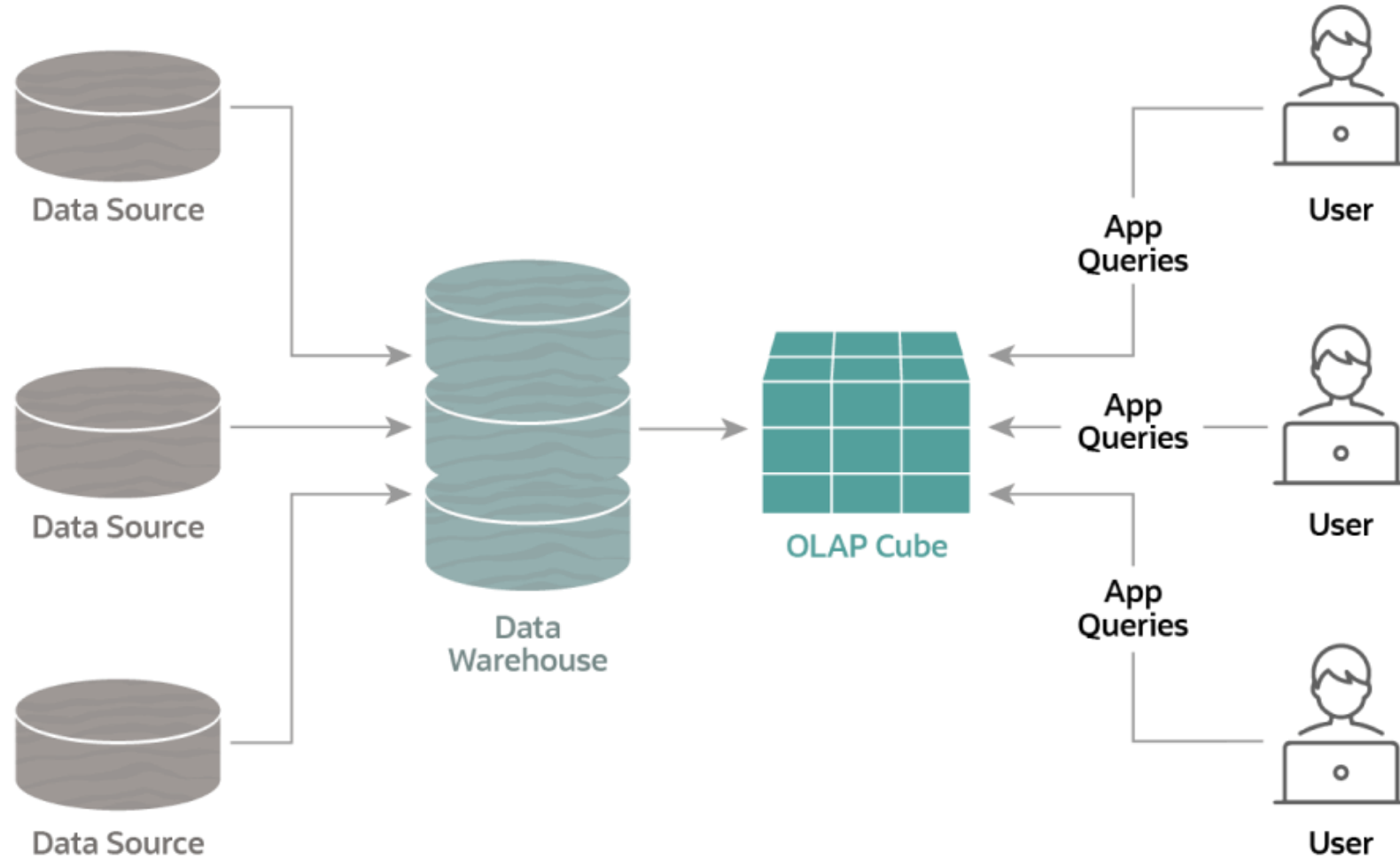


Learn

[Ver más](#)

The OLAP Process

How data is prepared for online analytical processing (OLAP)



El procesamiento analítico en línea (OLAP) transforma los datos relacionales de sistemas OLTP (procesamiento transaccional en línea) en estructuras multidimensionales.

MOLAP: OLAP multidimensional

OLAP sobre un cubo OLAP multidimensional.

ROLAP: OLAP relacional

Análisis multidimensional de datos que opera directamente sobre tablas relacionales, sin reorganizar primero los datos en un cubo. Las vistas son más estáticas y no pivotan.

Consultas SQL complejas en bbdd relacionales en DW.

HOLAP: OLAP híbrido

Divide la carga de trabajo entre MOLAP y ROLAP. Las tablas relacionales contienen mayores cantidades de datos, y los cubos OLAP se usan para agregaciones y procesamiento especulativo. Arquitectura compleja.

En 2026, las soluciones líderes combinan tecnologías tradicionales MOLAP (cubos precalculados) con ROLAP (consultas directas) y enfoques modernos de "Lakehouse".

Motores Tradicionales y Enterprise (MOLAP/ROLAP)

- **Microsoft SQL Server Analysis Services (SSAS):** Sigue siendo uno de los estándares del mercado, ofreciendo tanto modelos tabulares (más rápidos) como multidimensionales (cubos tradicionales).
- **Oracle OLAP:** Parte de la base de datos Oracle, optimizado para análisis complejo dentro de la misma plataforma de base de datos.
- **IBM DB2 / Cognos:** Utilizado comúnmente en grandes empresas para modelado analítico.

Motores OLAP de Alto Rendimiento y Tiempo Real (Modernos)

- **ClickHouse:** Base de datos columnar de alto rendimiento con ejecución vectorial, ideal para analítica en tiempo real a gran escala.
- **Apache Druid:** Diseñado para analítica de baja latencia con ingesta de datos en tiempo real, muy usado en time-series y eventos.
- **StarRocks / Apache Doris:** Motores OLAP modernos que combinan el rendimiento de cubos precalculados con la flexibilidad de SQL, ideales para analítica de datos en tiempo real y esquemas en estrella.
- **Apache Pinot:** Diseñado específicamente para analítica de usuario final con latencia ultra baja.

Soluciones de "Lakehouse" y Nube

- **Snowflake:** Ofrece capacidades OLAP potentes sobre data warehouses en la nube, permitiendo consultas analíticas complejas sin gestionar infraestructura.
- **Google BigQuery:** Motor de analítica de datos a gran escala que permite consultas OLAP rápidas sin necesidad de precalcular cubos tradicionales.
- **DuckDB:** Motor analítico embebido, rápido y sencillo, ideal para analítica local o incrustada (<10TB).

Motores Especializados e Inteligentes

- **icCube:** Motor OLAP in-memory de alto rendimiento especializado en cubos multidimensionales y análisis embebido.
- **Energent.ai:** Plataforma que utiliza IA para la creación y análisis interactivo de cubos OLAP.

La tendencia en 2026 es el uso de motores columnares y MPP (Massively Parallel Processing) que eliminan la necesidad de precalcular largos procesos de cubos, ofreciendo resultados al instante (ROLAP avanzado)

Esquema	Características	Pros	Contras	Cuándo Usarlo
Star Schema	Una tabla de hechos conectada a dimensiones sin normalización.	Rápido y fácil de consultar.	Puede tener redundancia de datos.	Análisis de datos rápido y eficiente.
Snowflake Schema	Dimensiones normalizadas en subdimensiones.	Optimiza almacenamiento, evita redundancia.	Más joins, menor rendimiento en consultas.	Cuando se necesita reducir duplicidad de datos.
Multidimensional Schema	Basado en cubos OLAP con múltiples dimensiones.	Permite análisis complejo con agregaciones.	Requiere más almacenamiento y procesamiento.	Cuando se usa OLAP y análisis avanzado.

Comentarios y preguntas